Frankfurt University
of Applied Sciences

– Faculty of Computer Science and Engineering –

Verbesserung eines Drohnenframes für FDM-3D-Druck und Integration von LED-Zustandsindikatoren

Mechatronikprojekt

Bachelor of Engineering B.Eng.

vorgelegt am 17.11.2026 von

Laurin Fehl

Matrikelnummer: 1409723

Praxisbeauftragter	:	Prof. Dr. Karsten Schmidt
Praxisreferat/Praxisreferentin	:	Adrian Hopfenspirger

Eidesstattliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellennachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Nutzung KI-gestützter Schreibwerkzeuge:

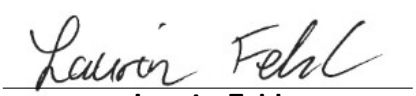
Ich habe zur Erstellung der Prüfungsarbeit KI-gestützte Schreibwerkzeuge und/oder andere KI-Anwendungen verwendet. Bei deren Einsatz habe ich steuernd gearbeitet. Ich versichere, dass mein eigener gestalterischer Einfluss in der Arbeit überwiegt. Sämtliche von mir verwendeten KI-Anwendungen habe ich als Hilfsmittel mit deren Produktnamen und Bezugsquelle (URL) angegeben und eine Übersicht der im Rahmen der Prüfungsarbeit genutzten Funktionen vollständig aufgeführt. Die von KI-gestützten Schreibwerkzeugen oder anderen KI-Anwendungen generierten Texte oder Textpassagen, die übernommen wurden, habe ich gekennzeichnet.

Eingesetzte KI-Werkzeuge:

Funktion	**KI-Anwendung**	**Abschnitt**	**Bezugsquelle**
Sprachliche Formulierung und Strukturierung von Textabschnitten	Claude (Sonnet 4.6, Opus 4.7) von Anthropic	Kapitel 1–6	`https://claude.ai`

Hinweis: Sämtliche fachlichen Inhalte, technischen Konzepte, Messungen, Implementierungen sowie alle inhaltlichen Entscheidungen stammen ausschließlich vom Verfasser. Die KI wurde ausschließlich zur sprachlichen Ausarbeitung und Formulierung eingesetzt. Alle generierten Texte wurden vom Verfasser inhaltlich geprüft, verantwortet und gegebenenfalls angepasst.

Frankfurt, 17.11.2026



Laurin Fehl

Inhaltsverzeichnis

1 Einführung **2**
1.1 Das FRoST-Labor . . . 2
1.1.1 Die DAEDALUS-Drohne . . . 2

2 Hintergrund **3**
2.1 Die European-Rover-Challenge . . . 3
2.1.1 Die Aufgaben des Wettbewerbs . . . 3
2.1.2 Anforderung und Rolle der Drohne . . . 4
2.2 Das FRoST-Labor . . . 4
2.3 Grundlagen 3D-Druck . . . 4

3 Stand der Technik **6**
3.1 Drohnenplattformen und Frames . . . 6
3.1.1 Bauformen von Multicoptern . . . 6
3.1.2 Vergleich von Quadcopter Framekonfigurationen . . . 7
3.1.3 Kommerzielle Frames und Bausätze . . . 7
3.2 Fertigung mit FDM-3D-Druck . . . 8
3.2.1 Funktionsprinzip FDM . . . 8
3.2.2 Materialien in Filamentform . . . 8
3.2.3 Konstruktionsrichtlinien . . . 9
3.2.4 Schwächen und Grenzen . . . 9
3.3 Zustandsanzeigen bei Drohnen . . . 10
3.3.1 Statusanzeige und Statemachine . . . 10
3.3.2 LED-Systeme und Ansteuerung . . . 10
3.3.3 Anforderungen und Normen . . . 10
3.4 Zusammenfassung und Ableitung . . . 10

4 Entwickelte Methoden **11**
4.1 Konstruktion des 3D-gedruckten Frames . . . 11
4.1.1 Konstruktionskonzept und Designentscheidungen . . . 11
4.1.2 Auslegung für den FDM-Druck . . . 13
4.1.3 Materialwahl . . . 13
4.1.4 Integration der Komponenten . . . 15
4.1.5 Fertigung und Montage . . . 15
4.2 LED-Statusanzeige . . . 15
4.2.1 Konzept und Zustandsdefinition . . . 15
4.2.2 Hardware . . . 19
4.2.3 Software: Zustandsautomat auf dem Companion Computer . . . 20
4.2.4 Anbindung an den Flugcontroller . . . 22
4.2.5 Software-in-the-Loop-Testumgebung . . . 23

5 Evaluation **27**
5.1 Vorgehen und Bewertungskriterien . . . 27

5.2 Evaluation des Frames . . . 27
5.2.1 Masse . . . 27
5.2.2 Maßhaltigkeit und Montage . . . 27
5.2.3 Mechanische Belastbarkeit . . . 27
5.2.4 Fertigungsaufwand . . . 28
5.2.5 Wartbarkeit . . . 28
5.2.6 Flugtests . . . 28
5.3 Evaluation der Statusanzeige . . . 28
5.3.1 Automatisierte Tests der Anzeigelogik . . . 28
5.3.2 Funktionstest an der Hardware . . . 29
5.3.3 Sichtbarkeit und Unterscheidbarkeit . . . 30
5.3.4 Übertragbarkeit auf den Flugbetrieb . . . 31

6 Ergebnisse **32**
6.1 Ergebnisse des Frames . . . 32
6.1.1 Masse . . . 32
6.1.2 Maßhaltigkeit und Montage . . . 32
6.1.3 Mechanische Belastbarkeit . . . 32
6.1.4 Fertigungsaufwand . . . 32
6.1.5 Wartbarkeit . . . 32
6.1.6 Flugtests . . . 32
6.2 Ergebnisse der Statusanzeige . . . 32
6.2.1 Automatisierte Tests der Anzeigelogik . . . 32
6.2.2 Funktionstest an der Hardware . . . 32
6.2.3 Sichtbarkeit und Unterscheidbarkeit . . . 33
6.2.4 Übertragbarkeit auf den Flugbetrieb . . . 33
6.3 Diskussion . . . 33
6.3.1 Erfüllung der Anforderungen . . . 33
6.3.2 Probleme und Grenzen . . . 33

7 Zusammenfassung und Ausblick **35**
7.1 Zusammenfassung . . . 35
7.2 Ausblick . . . 35

Literatur **36**

A Anhang **38**

Abbildungsverzeichnis

3.1 Quadcopter-Konfigurationen H, Plus und X mit Drehrichtung der Motoren [11] 7
3.2 Das Funktionsprinzip des Fused Deposition Modeling (FDM)-3D-Drucks 8
3.3 Beispielabbildung einer Maschinenampel 10

4.1 Render der Baugruppenansicht des neuen Drohnenframes 11
4.2 Belegung eines LED-Streifens am Beispiel eines linken Beins im autonomen Flug ohne Fehler. LED 0 zeigt die Seite an (links grün, rechts rot), LED 1–2 den Autonomiekanal, LED 3–4 den Statuskanal. 16
4.3 Aufbau der Software-in-the-Loop-Testumgebung der LED-Statusanzeige 24

Tabellenverzeichnis

2.1 Aus den ERC-Regeln abgeleitete Anforderungen an die Drohne 4

3.1 Vergleich der Quadcopter-Konfigurationen H, Plus und X 7

4.1 Übersicht der gedruckten Bauteile des Frames 12
4.2 Morphologischer Kasten der Filamente mit den Lösungsvarianten PET-G (grün), ASA (blau) und PA (orange) 14
4.3 Zuordnung von Drohnenzustand zu Anzeigemuster je Kanal 18
4.4 Zeitverhalten der Anzeigemuster 18
4.5 Verhalten der Anzeige bei fehlenden oder ungültigen Daten 19
4.6 Zuordnung der Landebeine zu den GPIO-Pins (BCM-Nummerierung) 19
4.7 Vom LED-Knoten ausgewertete Felder des Zustandsobjekts 21

5.1 Zuordnung der Anforderungen an die Statusanzeige zu Prüfung und Abnahmekriterium . . 29
5.2 Testmatrix für den Funktionstest der Statusanzeige in der Software-in-the-Loop (SIL)-Testumgebung 30

6.1 Ergebnisse des Funktionstests in der SIL-Testumgebung, Zeilennummern nach Tabelle 5.2 33
6.2 Erfüllung der Anforderungen nach aktuellem Stand 34

Abkürzungsverzeichnis

CAD Computer-Aided Design. 4, 5, 11

CBOR Concise Binary Object Representation. 21

DAEDALUS Deployable Aerial Exploration Drone for Autonomous Low-altitude Utility Scouting. 2

ERC European-Rover-Challenge. 2, 10, 15

ESC Electronic Speed Controller. 7

FDM Fused Deposition Modeling. iii, 4, 6, 8, 9

Fra-UAS Frankfurt University of Applied Sciences. 2

FRoST Frankfurt Robotics Science Team. 2–4, 6, 7, 9, 13, 14, 27

GPIO General Purpose Input/Output. 19, 20

IRCARUS ICRASUSlooong. 4, 11

LED Light-Emitting Diode. 15

MAVLink Micro Air Vehicle Link. 22

PIO Programmable Input/Output. 20, 21, 26, 31, 34

ROS Robot Operating System. 20, 28

SIL Software-in-the-Loop. iv, 15, 23, 28–31, 33, 34

UART Universal Asynchronous Receiver Transmitter. 22

Kurzfassung

Diese Arbeit beschreibt die Entwicklung eines verbesserten Drohnen-Frames mit Fokusierung auf 3D-Druck optimierte Konstruktion. Zusätzlich wird ein System aus LED-Indikatoren integriert, welche den aktuellen Flugmodus der Drohne anzeigen. Diese Flugmodusanzeige soll dem Regelwerk der European Rover Challenge entsprechen.

1 Einführung

1.1 Das FRoST-Labor

Das Frankfurt Robotics Science Team (FRoST)-Labor ist ein studentisches Team der Frankfurt University of Applied Sciences (Fra-UAS), welches Robotersysteme entwickelt. Gezielt werden Projekte für Bachelor- und Masterarbeiten ausgeschrieben um an der European-Rover-Challenge (ERC) teilzunehmen, ein internationaler Wettbewerb, bei dem Teams Marsrover bauen und in simulierten Missionen antreten. Aufgeteilt in sieben verschiedene Aufgaben ein selbstgebauter Marsrover, begleitet von einer Drohne, treten 25 Teams gegeneinander an.

Während das Team in vergangen Jahren fast hauptsächlich aus Mechatronik Master Studenten bestand, ist zur heutigen Zeit ein Mix aus Mechatronik Bachelor- und Masterstudenten vorhanden. Weitere Elektrotechnik Bachelorstudenten, sowie Austauschstudenten aus Vietnam ergänzen von Zeit zur Zeit das Team.

1.1.1 Die DAEDALUS-Drohne

Die Drohne Deployable Aerial Exploration Drone for Autonomous Low-altitude Utility Scouting (DAEDALUS) wurde aus Komponenten von einem Holybro X500 V2 kit gebaut. Bei der Drohennaufgabe des ERC-Wettbewerbs wird aus der Luft eine Fläche abgeflogen und mit Computer-Vision nach Proben gescannt, welche für eine spätere Untersuchung von seitens des Rovers markiert werden. Dabei ist ein stabiler und ruhiger Flug wichtig, bei der die Ausrichtung der Motoren, das Gewicht, Kameraposition und Vibrationsvermeidung eine große Rolle spielen.

2 Hintergrund

Um einen Überblick der Motivation, Aufbauweise und Integration zu geben, werden im folgenden die wichtigsten Punkte ausführlich beschrieben. Dabei werden der Wettbewerb, das FRoST-Labor, die Drohne und die Grundlagen behandelt.

2.1 Die European-Rover-Challenge

Jedes Jahr im September veranstaltet die AGH-University of Kraków in Kollaboration mit der European-Space-Foundation die European-Rover-Challenge. Dabei treten aus 124 Registrierungen aus 29 Ländern die besten 25 Teams gegeneinander an. Aufgeteilt in sieben Aufgaben wird das können der Teams mit ihrem Rover und der Drohne getestet.

2.1.1 Die Aufgaben des Wettbewerbs

Traverse Sub-Task
: Der Rover navigiert autonom entlang mehrerer Wegpunkte, ohne dass dem Operator Kamerabilder zur Verfügung stehen. Die Steuerung erfolgt allein über Telemetrie. Der autonome Modus muss durch ein eigenes Anzeigemuster der Statusleuchte erkennbar sein, und nach der Aktivierung darf sich der Rover erst nach einer kurzen Wartezeit bewegen. Während der Fahrt ist der Eingriff des Operators gesperrt.

Droning Sub-Task
: Als Luftaufklärung für den Rover erkundet die Drohne autonom mehrere Sektoren eines Drohnenkäfigs. Anhand von ArUco-Markern bestimmt sie, in welchem Bereich sie sich befindet und bestimmt mit Bilderkennung wo vordefinierte Proben auf den Boden liegen und markiert diese. Diese Aufklärung darf drei mal innerhalb der gesetzten Zeit wiederholt werden. nes Drehschalters, das

Maintenance Task
: Der Rover bedient mit seinem Manipulator ein Wartungspanel. Zu den Aufgaben gehören das Betätigen eines Hebel- und eiBedienen eines elektromagnetischen Schlosses und das vollständige Einstecken eines Netzsteckers in eine Buchse.

Probing Task
: Der Rover greift die vorgegebenen Probenobjekte (Probes), hält sie sicher und gibt sie kontrolliert wieder frei.

Surface Sampling
: Der Rover nimmt Regolith und Gesteinsproben auf und lagert sie in getrennten Behältern. Das Gewicht der Proben wird an Bord automatisch und möglichst genau bestimmt, ohne dass der Operator manuell eingreift. Anschließend werden die Proben ohne Verlust zur Ziellinie transportiert.

Deep Sampling
: Eine Bohreinheit entnimmt Bodenmaterial aus größerer Tiefe unterhalb der Oberfläche. Die entnommene Probenmenge muss eine Mindestmasse erreichen, die ebenfalls automatisch an Bord gemessen wird.

Astro-Bio Sub-Task

Der Rover handhabt einen Flüssigkeitsbehälter ohne Verschütten und positioniert ein pH-Messgerät an einer vorgegebenen Messstelle.

2.1.2 Anforderung und Rolle der Drohne

Beim Wettbewerb fliegt die Drohne in dem sogenannten Drohnenkäfig, ein 10x10x4 meter großer, mit Netzen an der Seite und einem Dach, abgeschirmten Quader. Durch EU-Richtlinien darf die Drohne nur in diesem Käfig autonom fliegen und sich frei bewegen. Der Drohnenpilot muss durchgehend Sichtkontakt halten und kann jederzeit eingreifen. Bei einem Notfallsignal oder Verbindungsverlust muss die Drohne selbstständig landen. Während der Akku zwischen den drei Versuchen getauscht werden darf, profitiert das Team von einem leichten und stabilen Aufbau des Frames um so Zeit zu sparen.

Aus den Regeln des Wettbewerbs ergeben sich für die Drohne die in Tabelle 2.1 zusammengefassten Anforderungen.

Tabelle 2.1: Aus den ERC-Regeln abgeleitete Anforderungen an die Drohne

Thema	Anforderung	Konsequenz für diese Arbeit
Sicherheit	Pilot kann jederzeit eingreifen; automatische Landung im Notfall	Zugänglichkeit der Elektronik im Frame
Anzeige	Zustand vor Bewegung und im autonomen Modus erkennbar	LED-Statusanzeige (Abschnitt 4.2)
Masse	Obergrenze für die Gesamtmasse	Leichtbau-Frame im FDM-Druck
Betrieb	Einsatz im Drohnenkäfig, wechselnde Lichtverhältnisse	Sichtbarkeit der LEDs, robuste Konstruktion

2.2 Das FRoST-Labor

Wie in Kapitel 1 erklärt, ermöglicht FRoST ein erarbeiten von Projekten für Module der Studiengänge. Studierende profitieren dabei von der bereitgestellten Hardware und der Möglichkeit vor Ort mit anderen Kommilitonen zu arbeiten.

Der Vorgänger ICRASUSlooong (IRCARUS) diente zum entdecken und experimentieren neuer Technik und Software. Der alte Frame wurde aus geschnittenen Karbonplatten und -rohren zusammengebaut. Durch das große und schwere Design wurde jedoch ein neuer Frame benötigt der als Mechatronik-Projekt ausgeschrieben wurde.

2.3 Grundlagen 3D-Druck

Beim 3D-Druck wird ein Bauteil nicht aus einem Block herausgearbeitet, sondern Schicht für Schicht aufgebaut. Man spricht deshalb auch von additiver Fertigung. Aus einem digitalen Computer-Aided Design (CAD)-Modell berechnet eine Software die einzelnen Schichten, die der Drucker anschließend nacheinander herstellt.

In dieser Arbeit kommt das FDM-Verfahren zum Einsatz. Dabei wird ein Kunststoffdraht (Filament) in einem beheizten Druckkopf geschmolzen und in dünnen Bahnen auf eine Bauplattform aufgetragen. Das Material kühlt ab, erstarrt und verbindet sich mit der darunterliegenden Schicht. Das Verfahren ist vergleichsweise günstig und erlaubt es, auch komplexe Formen und Hohlräume herzustellen, die sich klassisch

nur schwer fertigen lassen. Zudem können Bauteile schnell geändert und neu gedruckt werden, was für die Weiterentwicklung eines Drohnenframes vorteilhaft ist.

Vor jedem Drucken muss das CAD-Modell in einer Slicer-Software in einzelne Schichten zerlegt werden. Dabei werden Parameter wie Schichthöhe, Füllung, Wandstärke und Stützstrukturen festgelegt. Die Software erzeugt daraus einen G-Code, der die Bewegungen des Druckkopfes, der Bauplattform und die Extrusion des Materials steuert.

Gedruckte Teile haben jedoch Eigenheiten, die bei der Konstruktion beachtet werden müssen. Ihre Festigkeit hängt unter anderem vom verwendeten Material und von der Ausrichtung beim Drucken ab. Auf diese Aspekte wird im Kapitel 3 genauer eingegangen.

3 Stand der Technik

Dieses Kapitel bearbeitet den aktuellen Stand der Technik in den Bereichen, die für diese Arbeit relevant sind. Zunächst werden Bauformen und Konstruktionsweisen von FDM -3D-Druck-Bauteilen betrachet, sowie eine Zustandsaufnahme der Software der Drohne um die Ausmaße der benötigten Arbeit abzuleiten.

3.1 Drohnenplattformen und Frames

Der Markt von selbst gebauten Drohnen ist in den letzten Jahren stetig gewachsen. Flight Controller und ESC Boards werden von Jahr zu Jahr günstiger, variantenreicher und leistungsstärker. In dieser Sektion werden die wichtigsten Faktoren von selbstgebauten Drohnen erarbeitet und analysiert. Dabei werden von FRoST vorher bestimmte Möglichkeiten bearbeitet.

3.1.1 Bauformen von Multicoptern

Konträr zu dem ready-to-fly kaufbaren Drohnen kann bei einem selbst gebauten Design die freie Wahl der Konfigurationen getroffen werden. Dabei muss auf den Zielnutzen geachtet werden, da verschiedene Konfigurationen unterschiedliche Vor- und Nachteile besitzen. Die Framekonfiguration definiert wie die einzelnen Motoren zueinander positioniert sind. Bei der Auswahl der Framekonfiguration ist es auch wichtig zu beachten wie viele Motoren benutzt werden. Die Anzahl der Propeller entscheidet über wichtige Charakteristika für den Zielnutzen. Im Folgenden werden die beiden für diese Arbeit relevanten Bauformen, der Hexacopter und der Quadcopter, in Anlehnung an [13] gegenübergestellt.

Hexacopter
: Das Hex im Namen steht für die Zahl sechs und damit für die Anzahl der Motoren. Die Last der Drohne verteilt sich gleichmäßig auf sechs Propeller, wodurch jeder einzelne Rotor weniger belastet wird. Mit einem Eigengewicht von etwa 2,3 kg bis 6,8 kg kann ein Hexacopter eine Nutzlast von etwa 2,3 kg bis 4,5 kg tragen und eignet sich damit für schwere Ausrüstung wie professionelle Kamerasysteme. Durch die zusätzlichen Rotoren liegt er auch bei Wind ruhig in der Luft. Der größte Vorteil ist die Ausfallsicherheit: Fällt ein Rotor aus, verliert der Hexacopter nur etwa 16 % seines Schubs und kann in der Regel noch sicher zurückfliegen. Dem stehen eine mit etwa 10 min bis 20 min eher kurze Flugzeit ohne Last, eine geringere Agilität sowie höhere Anschaffungs- und Ersatzteilkosten gegenüber. Hexacopter werden daher vor allem im professionellen Bereich eingesetzt.

Quadcopter
: Beim Quadcopter verteilt sich die Last auf nur vier Propeller, sodass jeder Rotor einen größeren Anteil tragen muss. Dafür kann dieser deutlich leichter als ein Hexacopter sein, kann allerdings auch nur eine entsprechend geringere Nutzlast aufnehmen. Durch das geringere Gewicht kann er eine längere Flugzeit erreichen und ist meistens schneller und wendiger. Bei Wind ist er dagegen unruhiger als ein Hexacopter. Der entscheidende Nachteil liegt in der fehlenden Redundanz: Der Ausfall eines Rotors bedeutet einen Schubverlust von 25 % und führt meist zum Absturz. Im Gegenzug sind Quadcopter günstiger in der Anschaffung, Ersatzteile sind preiswert und leicht erhältlich, und Reparaturen lassen sich meist selbst durchführen. Zudem ist ein Quadcopter einfacher zu fliegen.

3.1.2 Vergleich von Quadcopter Framekonfigurationen

FÜr die Drohne bei FRoST wurde ein Quadcopter Design ausgwählt. Bei diesen jedoch gibt es die verschiedenen Framekonfigurationen Quad H, Quad + und Quad X. Der grundlegende Unterschied liegt in der Anordnung der Arme in Relation zum Körper der Drohne und den Motoren. Um einen Überblick über die Unterschiede zu geben, werden in Tabelle 3.1 die wichtigsten Unterschiede zusammengefasst. Für eine visuelle Stütze sind in Abbildung 3.1 die Konfigurationen dargestellt. Dabei wird sich auf die Artikel [11] und [7] bezogen.

Tabelle 3.1: Vergleich der Quadcopter-Konfigurationen H, Plus und X

	Quad H	**Quad +**	**Quad X**
Armanordnung	Zwei Querträger an einem Längsrumpf	Arme in und quer zur Flugrichtung	Arme diagonal zur Flugrichtung
Nick-/Rollsteuerung	Geringer	Geringer	Ca. 30 % höher
Besonderheiten	Lange Basis, mehr Platz für Akkus/Technik	Propeller im Sichtfeld von Kameras	Einfacher Zusammenbau und Reparatur
Typischer Einsatz	FPV-Racing	Bildung	Ready-to-Fly Drohnen

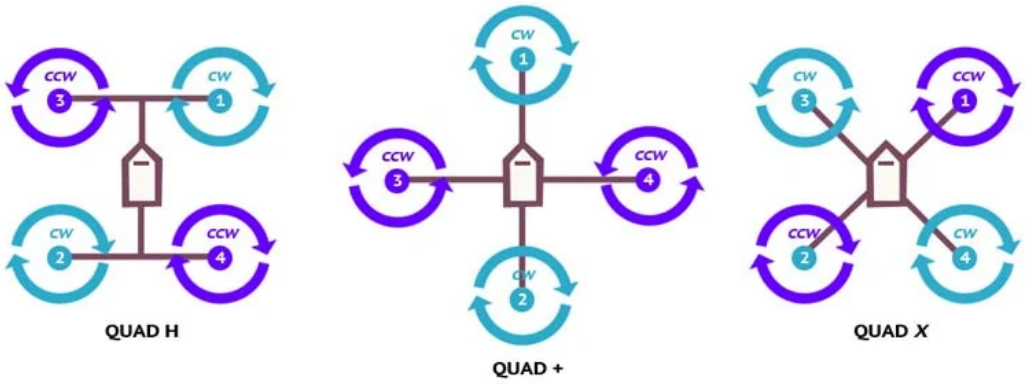


Abbildung 3.1: Quadcopter-Konfigurationen H, Plus und X mit Drehrichtung der Motoren [11]

3.1.3 Kommerzielle Frames und Bausätze

Der Markt für selbst gebaute Drohnenkits ist in den letzen Jahren stark gewachsen. Auch wenn Kits eher in Richtung FPV-Racing und Hobbybereich gehen, gibt es weitere Standards "Development Kitsffür den professionellen Bereich. Dort werden elektronische Komponenten wie Flight Controller, Electronic Speed Controller (ESC) und Motoren auf Leistung und Kompatibilität abgestimmt. Diese Frames benutzen meist Carbonplatten und -rohre, um ein möglichst leichtes und steifes Chassis zu erhalten. Für dei Drohne bei FRoST wurde das PPX4 Development Kit - X500 v2"von Holy Bro augewählt. Die ist ein Quadcopter im Quad X-Design, welches viel Erweiterbar ist und eine gute Basis bietet.

Bei FPV orientierten Drohnenkits sind 3D-gedruckte Frames oft eine günstigere Alternative zu Carbonrahmen und gewinnen an Beliebtheit. Dort werden oft brechende Teile wie Motorträger 3D-gedruckt, um einfache Ersatzteile zu haben.

3.2 Fertigung mit FDM-3D-Druck

FDM-3D-Druck ist eine Fertigungstechnologie, welche in den letzten Jahren stark gewachsen ist, im Hobby- sowie industriellem Bereich. In dieser Sektion werden die Funktionsweise, die Materialien und die Konstruktionsrichtlinien für FDM-3D-Druck-Bauteile betrachtet.

3.2.1 Funktionsprinzip FDM

Das Kernprinzip der FDM-Technologie ist das fertigen eines Werkstücks in vielen einzelnen Schichten, welche durch eine Düse aufgetragen werden. Dabei kann eine vielzahl an thermoplastischen Kunststoffen verwendet werden, welche in Filamentform, also als langen Faden, vorliegen. Dieses Filament wird durch einen Motor in die Düse gedrückt, dort erhitzt und auf die Bauplattform aufgetragen. Der FDM-3D-Druck ist ein additives Fertigungsverfahren im Vergleich zu subtraktiven Verfahren wie Fräsen oder Drehen. Dies ermöglicht komplexe Geometrien und Hohlräume, die mit traditionellen Methoden schwer bis unmöglich herzustellen wären. Abbildung 3.2 zeigt das Funktionsprinzip des FDM-3D-Drucks.

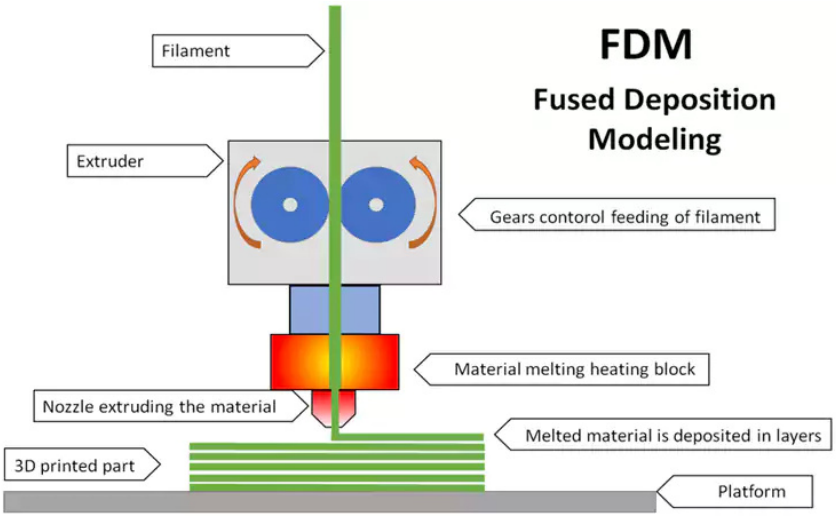


Abbildung 3.2: Das Funktionsprinzip des FDM-3D-Drucks

3.2.2 Materialien in Filamentform

Durch den Einsatz von FDM-3D-Druck auch im Hobbybereich, gibt es eine Vielzahl von Materialien, welche schon vorgefertig in Filamentform erhältlich sind. Diese reichen von einfachen und schwachen bis stabilen und temperaturresistenten Kunststoffen. Da jeder Kunststoff seine eigenen Eigenschaften hat, ist sinnvoll einen Überblick über die gängigsten Materialien sich zu verschaffen.

PLA

Der Kunststoff PLA, Polylactid acid, ist ein Kunststoff welcher aus nachwachsenden Rohstoffen wie Maisstärke ode Zuckerrohr hergestellt wird. PLA ist ein sehr einfacher Kunststoff, welcher sich für den Einstieg in den 3D-Druck eignet, da dieser besonders leicht zu drucken ist. Seine leichtigkeit im Druck macht es jedoch zu einem nicht besonders temperaturbeständigen Kunststoff. PLA ist zudem ein spröder Kunststoff, welcher bei Belastung schnell bricht.

PET-G

Der Kunststoff PET-G, Polyethylenterephthalat Glycol, ist der beliebte Kunststoff PET mit dem Zusatz Glycol. Dieser Zusatz macht den Kunststoff zäher und weniger spröde. Er ist ein wenig aufwendiger zu drucken als PLA, jedoch genauso beliebt und weit verbreitet.

ABS
Der Kunststoff ABS, Acrylonitrile Butadiene Styrene, ist ein sehr beliebter Kunststoff der Industrie. Er ist zäh und temperaturbeständiger als PLA und PETG, jedoch auch weitaus schwieriger zu drucken, da sich der Kunststoff beim Abkühlen stark zusammenzieht und sich verzieht. ABS ist schwach gegenüber UV-Strahlung und kann bei längerer Sonneneinstrahlung spröde werden. Auch ist der Kunststoff gesundheitlich bedenklich, da beim Drucken giftige Dämpfe entstehen.

ASA
Der Kunststoff ASA, Acrylonitrile Styrene Acrylate, ist ein Kunststoff der dem ABS sehr ähnlich ist, jedoch UV-beständiger und besser für den Außeneinsatz sich eignet. Es ist genau so fast genauso schwierig zu drucken wie ABS. ASA ist genau wie ABS gesundheitlich bedenklich, da beim Drucken giftige Dämpfe entstehen.

PA Der Kunststoff PA, Polyamide, ist ein sehr stabiler und temperaturbeständiger Kunststoff. Er ist besonders geeignet für Anwendungen, bei denen hohe mechanische Belastungen auftreten. Da dieser Kunststoff sehr hygroskopisch ist, muss er vor dem Drucken getrocknet werden. Zudem ist im Hobbybereich oft nur PA Zusätzen wie Kohlenstofffasern pder Glasfasern erhältlich, da PA sich auch beim abkühlen stark zusammenzieht, sogar schon während des Druckens.

TPU
Der Kunststoff TPU, Thermoplastic Polyurethane, ist ein sehr flexibler Kunststoff, welcher sich für Anwendungen eignet, bei denen Elastizität und Flexibilität benötigt wird. TPU ist ein sehr schwieriger Kunststoff zu drucken. TPU nimmt auch sehr schnell Feuchtigkeit auf, sodass dieser vor dem Drucken getrocknet werden muss.

3.2.3 Konstruktionsrichtlinien

Bedingend der Schichtbauweise des FDM-3D-Drucks, haben Bauteile eine Anisotropie, also unterschiedliche mechanische Eigenschaften in verschiedenen Richtungen. Die Festigkeit eines Bauteils hängt stark von der Ausrichtung der Schichten ab. Daher ist es wichtig, die Orientierung des Bauteils während des Druckens zu berücksichtigen, um die gewünschte Festigkeit zu erreichen. Zudem müssen Einstellungen im Slicer wie Wandstärke, Füllung und Stützstrukturen beachtet werden, um ein qualitativ hochwertiges Bauteil zu erhalten.

Die Orientierung des Bauteils ist auch schon während der Konstruktion zu berücksichtigen, um so keine Flächen zu erzeugen, welche nur schwer oder gar nicht zu drucken sind. Zudem ist es wichtig, die Belastungspunkte des Bauteils zu kennen, um die Schichten in der richtigen Richtung auszurichten.

3.2.4 Schwächen und Grenzen

Durch die erarbeiten Materialcharakteristika in 3.2.2 können die Schwächen und Grenzen der Filamente beim Konstruieren berücksichtigt werden. Schichtrichtung und Wandstärke sind eintscheidene Faktoren für die Festigkeit eines Bauteils. Zudem ist es wichtig, die Belastungspunkte des Bauteils zu kennen, um die Schichten in der richtigen Richtung auszurichten. Beim konstruieren und nicht beachten dieser Faktoren kann es zu den typischen Schwächen von FDM-3D-Druck-Bauteilen kommen. Dazu gehören eine schlechte Schichthaftung, Vibrationen und Temperaturbeständigkeit.

Im FRoST-Labor standen zum Zeitpunkt dieser Arbeit zwei 3D-Drucker vom Typ Bambu Lab X1-Carbon zur Verfügung. Ihre maximale Drucktemperatur beträgt 300 °C, eine aktive Bauraumheizung ist nicht vorhanden. Daraus ergibt sich eine weitere Schwäche: Bei zu niedriger Temperatur können Schwachstellen im Druck entstehen.

3.3 Zustandsanzeigen bei Drohnen

Gemäß den Anforderungen des ERC Wettbewerbs muss die Drohne eine Möglichkeit besitzen, welche den Status und Zustand anzeigt. In dieser Sektion werden die Möglichkeiten der Statusanzeige betrachtet und die Anforderungen an diese abgeleitet.

3.3.1 Statusanzeige und Statemachine

In der Industrie werden Maschinen und Anlagen oft mit Statusanzeigen ausgestattet, um den Zustand der Maschine anzuzeigen. Oft werden dabei Maschinenampel, auch Signalsäulen genannt, wie im Bild 3.3 benutzt. Diese bestehen aus mehreren Leuchten, welche eine bestimmte Farbe und Blinkfrequenz für einen bestimmten Zustand der Maschine darstellen. Die Drohne ist nicht gleichzusetzen mit einer Maschine, jedoch ist es sinnvoll, die gleiche Logik zu verwenden. Die Drohne kann in verschiedene Zustände wechseln, welche durch die Statusanzeige dargestellt werden. Dabei ist es wichtig, dass die Zustände eindeutig und leicht verständlich sind. Der Companion Computer der Drohne bestimmt die Zuständer und steuert die Statusanzeige. Die Zustände der Drohne werden in einer Statemachine dargestellt, welche die Zustände und Übergänge zwischen den Zuständen definiert.



Abbildung 3.3: Beispielabbildung einer Maschinenampel

3.3.2 LED-Systeme und Ansteuerung

Moderne LED-Systeme wie NeoPixel oder WS2812B ermöglichen es, eine Vielzahl von Farben und Effekten mit nur einem Datenpin zu steuern, wodurch eine einfache Darstellung von Zuständen und Warnungen möglich ist. Die Ansteuerung erfolgt über den Companion Computer der Drohne, wo durch eine eigenene Controllersoftware die LEDS ansteuern kann, um unterschiedliche Zustände der Drohne darzustellen.

Die LED-Streifen von WS2812B benötigen nur drei Pins für die Ansteuerung: einen für die Datenleitung, einen für die Stromversorgung und einen für die Masse. Ein Vorteil von LED-Streifen ist, dass nur eine Seite verbunden werden muss, um die gesamte Kette zu steuern. Die Datenleitung wird von LED zu LED weitergegeben, wodurch eine einfache Verkabelung möglich ist. Diese Streifen werden auch aRGB genannt, a für addressable, um jede einzelne LED individuell ansteuern zu können.

3.3.3 Anforderungen und Normen

Die Anforderungen an die Statusanzeige der Drohne ergeben sich aus den Regeln des ERC-Wettbewerbs, welche in Tabelle 2.1 im Kapitel 2 zusammengefasst sind. Die Anzeige muss den Zustand der Drohne eindeutig darstellen, sodass der Operator jederzeit erkennen kann, in welchem Zustand sich die Drohne befindet. Zudem wurden eigene Anforderungen definiert, wie dedizierte LEDS, welche links grün und rechts rot leuchten sollen, um so die Orientierung der Drohne leichter erkennen zu können.

3.4 Zusammenfassung und Ableitung

Durch ein aufarbeiten der vorhanden Technik und Mittel die zur Verfügung stehen, kann so in einem festgelegten Rahmen die Arbeit durchgeführt werden um klare Ergbenisse erarbeiten zu können. Durch Befunde aber auch während der Entwicklung werden altenativ Lösungen gesucht die auch das Ziel erfüllen würden.

4 Entwickelte Methoden

In diesem Kapitel werden die Entwickelten Methoden aufgearbeitet und im Detail beschrieben. Zuerst wird auf die Konstruktion des neuen Frames eingegangen. Welche Design entscheidungen getroffen wurden und wie diese Umgesetzt wurden. Danach wird die Entwicklung des LED-Controllers beschrieben, sowie die Einbindung der Software.

4.1 Konstruktion des 3D-gedruckten Frames

Die erste Aufgabe dieser Arbeit ist die Entwicklung eines neuen Frames für die dDrohne. Die Konstruktion des neuen Frames erfolgte im CAD-Programm Autodesk Inventor. Die Vorgängerversion IRCARUS liegt ebenfalls als Baugruppendatei in diesem Programm vor und diente als Referenz für bestehende Maße sowie für Komponenten mit festgelegten Abmessungen, wie beispielsweise die Batterie oder den Flightcontroller. Die einzelnen Bauteile wurden jeweils als .ipt-Dateien erstellt und gespeichert und anschließend zu einer Baugruppe zusammengefügt, wie in Abbildung 4.1 dargestellt.



Abbildung 4.1: Render der Baugruppenansicht des neuen Drohnenframes

In Tabelle 4.1 sind alle konstruierten Bauteile aufgelistet und kurz erklärt. Für die Teile baseplate_top und baseplate_bottom, sowie motor_link_left und motor_link_right wurden Zeichnungen als Vorlage zum Ableiten der Teile benutzt. So konnte durch die Änderung einer Bemaßung in nur einer Zeichnung, beide Bauteile automatisch neu anpassen.

4.1.1 Konstruktionskonzept und Designentscheidungen

Um die ersten Teile konstruieren zu können, musste eine Zielmaß, bzw. eine gewünschte Größe ausgesucht werden, welche die Drohne haben sollte. Das Ziel des neuen Frames war auch neben der Verbesserung des Gewichtes, eine Reduzierung des Fußabdruckes der Drohne. Der Abstand zwischen den Propellerblättern wurde verringert auf ein angemessenes Minimun. Durch eine Verkleinerung des Abstand beider Motorverbindungen von der baseplate zu den Motorhaltern hin, ergibt sich eine Schräge welche passend auf den Abstand der Motoren ausgerechnet wurde. Wie in Abbildung (hier ref) zu sehen ist, ergibt sich aus der

Tabelle 4.1: Übersicht der gedruckten Bauteile des Frames

Bauteil	Beschreibung
Baseplate	
cover	Die Elektronik, also Flight Controller und Raspberry Pi, mussten durch fliegende Teile oder Spritzwasser geschützt werden. Die Abdeckung ist kein strukturelles Bauteil und wird deshalb nur mit Magneten an dem Roundframe befestigt.
roundframe	Um an der Drohne weitere Bauteile und Komponenten zu befestigen, wurde ein Ring um die Grundplatte gelegt, welcher zu jeder der vier Seiten, vier Einschmelzgewinde bietet. So können je nach Designentscheidung bis zu zwei Komponenten extra an einer Seite befestigt werden.
link_bottom	Um die Verbindungen zu den Motoren stabil zu designen, musste ein oberer und untere Link konstruiert werden. Der untere Link benutzt Einschmelzgewinde um mit dem oberen Link zusammengeschraubt zu werden.
link_top	Der obere Link besitzt Löcher für versenkte Zylinderkopfschrauben damit die Halterung für den Link symmetrisch bleibt.
gps_mount	Das vorhande GPS-Modul besitzt eine Halterung welche die sensiblen Sensoren mit mehr Abstand montiert. Dieses Bauteil wurde auf die existierende Halterung angepasst.
battery_holder	Die Batteriehalterung ist ein Bauteil welches doppelt gedruckt werden kann, da es symmetrisch designed wurde. Zum Festhalten der Batterie werden zwei Klettverschlusslaschen benutzt, welche durch einfache Durchführungen im Bauteil sich halten.
optical_mount	Die halterung für den Optical-Flow-Sensor ist besonders stabil designed, da durch ein verfälschen der Werte dieses Sensor, die Flugstabilität deutlich beinflusst werden würde.
picam_mount	Die Halterung für die Raspberry Pi Kamera wurde extra für eine seitliche Montage der Drohne designed, da vorne und hinten schon durch den Optical-Flow-Sensor und dem GPS blockiert war. Eine bestimmte Ausrichtung oder Orientierung der Kamera ist nicht nötig, da durch Software alles korrigiert werden kann.
Motor	
motor_plate	Durch den stark erwärmenden Motor, wurde eine Platte zum dazwischen legen konstruiert, welche die Halterung des Motors schützen sollte.
motor_mount	Die Halterung des Motors ist eins der wichtigsten und am meisten belasteten Bauteile. Dadurch wurde eine stabile Konstruktion erfordert, welche auch bei Belastung nicht direkt brechen würde.
motor_foot	Die Füße der Drohne sind direkt unter dem Motoren befestigt. Das gebogene Design kann Energie im Fall einer harten Landung aufnehmen. Zudem sind auf der Innenseite Kanäle konstruiert umd LED-Streifen mit durchsichtigem doppelseitigen Klebeband zu befestigen für die Statusanzeige der Drohne über LEDS.
Verbindung	
motor_link_left	Die linke Version der Verbindungsarme von der baseplate zu den motor_mounts beinhaltet Löcher für versenkte Zylinderkopfschrauben an der Motorenseite.
motor_link_right	Die rechte Version beinhaltet die Löcher für die Einschmelzgewinde an der Motorseite. Beide

Symmetrielinie der Arme ein Dreieck wo der Winkel α durch den Sinus berechnet werden kann. Abbildung (hier ref) zeigt das entstandene Dreieck und Formel (hier ref) berechnet den Winkel in Grad. Beim Anpassen der Motorarme, also eine Veränderung der Hypetnuse des Dreiecks, musste dann in der Ableitungsskizze der base_link top und bottom, sowie in der Motorhalterung geändert werden.

4.1.2 Auslegung für den FDM-Druck

Die Auslegung der Bauteile um die optimale Stabilität der vorgegebenen 3D-Druck-Geometrie führte zu sorgfältig ausgewählten besonderen Designentscheidungen. Diese Entscheidung können an drei Beispielen besonders klar dargstellt werden.

Motorarme:
Die Motorarme wurden im Vergleich zu den anderen Bauteilen liegend gedruckt, damit die Schichtlinien nicht in Richtung der am meisten aufzunehmenden Kraft liegen. Dadurch konnten auch Löcher für Schrauben so ausgelegt werden, dass die meiste Kraftaufnahme den Vorteil der Schichtrichtung nutzen konnte. (hier vielleicht zeichnung von cad screenshot und wirkende kräfte)

Motorhalter:
Bei den Motorhaltern wurde ein spezieller Designtrick benutzt um den Stabilitätspin, an der Stelle zum montieren der Füße, besonders bruchsicher zu gestalten. Dazu wurde nicht normal ein Kreis von der Fläche aus extruiert, sondern davor ein größere Kreis ausgeschnitten um 2 mm. In diesem Auschnitt wurde der Pin dann extruiert und mit Rundungen die Kanten abgerundet. Durch die Konstruktionsbedingte Richtung für die Befestigung der Motoren, entstehen im schlimmsten Fall an diesem Stabilitätspin Scherkräfte. (hier noch bild von schnittansicht)

Motorfuß:
Der Motorfuß ist im Vorfeld schon auf ein bestimmtes Material bedacht designed worden. Durch die Aufgabe, die Energie bei einer härteren Landung aufnehmen zu müssen, aber auch diese Energie nicht wieder zurückzugeben, ist das Material TPU ausgewählt worden. Ein durchsichtiges TPU war zur Zeit der Konstruktion im Bestand des FRoST-Labors, weshalb entschieden wurde dieses zu benutzen und davon ab zu designen.

4.1.3 Materialwahl

Die in Abschnitt 3.2.2 erarbeiteten Materialien dienten als Grundlage, um das passende Filament zu finden. Die Auswahl wurde mit einem morphologischen Kasten getroffen, der in Tabelle 4.2 zu sehen ist. Er ordnet jedes Filament je Merkmal einer Ausprägung zu. Eine hohe Ausprägung ist dabei nicht immer ein Vorteil, zum Beispiel bei Materialkosten oder Druckaufwand. Die drei hervorgehobenen Filamente bilden die Lösungsvarianten.

Tabelle 4.2: Morphologischer Kasten der Filamente mit den Lösungsvarianten PET-G (grün), ASA (blau) und PA (orange)

Merkmal	**Gering**	**Mittel**	**Hoch**
Mechanische Belastbarkeit	PLA, TPU	**PET-G**	ABS, **ASA**, **PA**
Zähigkeit	PLA	**PET-G**, ABS, **ASA**	**PA**, TPU
Temperaturbeständigkeit	PLA, TPU	**PET-G**	ABS, **ASA**, **PA**
UV-Beständigkeit	PLA, ABS	**PET-G**, **PA**, TPU	**ASA**
Feuchtigkeitsaufnahme	PLA, ABS, **ASA**	**PET-G**	**PA**, TPU
Dichte	ABS, **ASA**, **PA**	TPU	PLA, **PET-G**
Druckaufwand	PLA, **PET-G**	ABS, **ASA**	**PA**, TPU
Emissionen beim Druck	PLA, **PET-G**, TPU	**PA**	ABS, **ASA**
Materialkosten	PLA, **PET-G**	ABS, **ASA**, TPU	**PA**

Aus dem morphologischen Kasten gehen drei Lösungsvarianten für unterschiedliche Anforderungen hervor:

PET-G: günstige Abdeckungen
PET-G eignet sich am besten für Abdeckungen und andere nicht tragende Bauteile. Es ist günstig und einfach zu drucken und dabei zäher und temperaturbeständiger als PLA. Für den gesamten Frame reichen seine mechanische Belastbarkeit und Temperaturbeständigkeit jedoch nicht aus.

ASA: günstiger und stabiler Frame
ASA ist die beste günstige Lösung für den gesamten Drohnenframe. Es verbindet eine hohe Belastbarkeit und Temperaturbeständigkeit mit geringer Dichte und mittleren Materialkosten. Gegenüber dem ähnlichen ABS ist es deutlich UV-beständiger. Wegen der Emissionen muss beim Druck auf eine gute Belüftung geachtet werden.

PA: bester, aber teurer Frame
PA ist die leistungsstärkste Lösung für den gesamten Drohnenframe, da es hohe Belastbarkeit, Zähigkeit und Temperaturbeständigkeit vereint. Dem stehen hohe Materialkosten und ein hoher Druckaufwand gegenüber. Zudem nimmt PA stark Feuchtigkeit auf und muss vor dem Druck getrocknet werden.

Nach dem morphologischen Kasten eignen sich sowohl ASA als auch PA für den Drohnenframe. Gewählt wurde schließlich ein carbonfaserverstärktes PA11-Filament (PA11-CF-SL) der Firma Xenia, welches als Sponsoring zur Verfügung gestellt wurde. Damit entfällt der größte Nachteil, die hohen MAterialksoten, von PA. PA11 unterscheidet sich von anderen Polyamiden wie PA6 oder PA12 dadurch, dass es nicht auf Erdöl basiert, sondern pflanzlich aus Rizinusöl hergestellt wird. Die eingearbeiteten Carbonfasern erhöhen die Steifigkeit des Materials und verringern zugleich das in Abschnitt 3.2.2 beschriebene starke Schrumpfen von PA beim Abkühlen. Damit lassen sich auch größere Bauteile wie die Baseplate maßhaltig drucken.

Ein weiterer Vorteil des Filaments ist seine geringe Dichte. Durch die „UltraLite“-Rezeptur von Xenia erreicht das Filament eine Dichte von nur $0{,}99\,\mathrm{g/cm^3}$, während ASA mit $1{,}07\,\mathrm{g/cm^3}$ etwa 8 % schwerer ist. Bei gleichem Bauteilvolumen ist der Frame aus PA11-CF damit leichter als ein vergleichbarer Frame aus ASA. Für eine Drohne ist jedes eingesparte Gramm relevant, da sich ein geringeres Gewicht direkt auf Flugzeit auswirkt. Die übrigen Nachteile von PA bleiben jedoch bestehen: Das Filament nimmt schnell Feuchtigkeit auf und muss vor dem Druck getrocknet werden. Zudem wirken die Carbonfasern abrasiv, weshalb eine gehärtete Düse benötigt wird. Dies konnte jedoch im FRoST-Labor mit den bestehenden Druckern und Filamenttrockern, ohne extra Arbeit umgesetzt werden.

4.1.4 Integration der Komponenten

Der Flightcontroller und der Raspberry Pi wurden auf der Platte mit einer Halterung befestigt, welche besonders Platzsparend konstruiert wurde. So wurde der Raspberry Pi über den FC in einem kleinen Winkel angebracht, sodass die USB-Anschlüsse an der Hinterseite leichter zugänglich waren.

4.1.5 Fertigung und Montage

Durch die Absprache mit Xenia als Filamentsponsor, konnten Druckparameter und -profile ausgestauscht und übernommen werden. Es wurde jeglich die Düsentemperatur erhöht, da diese bei jedem Drucker selber angepasst werden muss.

Zur Nachbearbeitung der 3D-Druck-Teile gehörte das Installieren von Einschmelzgewinden. Diese wurden mit einem speziellen Aufsatz für diese in die dafür vorgesehen Löcher eingesetzt. Dabei muss auf eine ordentliche und vorsichtige installation geachtet werden, da ein schief eingeschmozlenes Gewinde, keine Schraube annehmen kann.

4.2 LED-Statusanzeige

Neben dem Frame ist die zweite Aufgabe dieser Arbeit eine Statusanzeige, die den Zustand der Drohne von außen erkennbar macht. Dazu wurden an den vier Landebeinen adressierbare Light-Emitting Diodes (LEDs) angebracht, die vom Companion Computer der Drohne angesteuert werden. In diesem Abschnitt werden zunächst das Anzeigekonzept und die Hardware beschrieben, anschließend die Software auf dem Companion Computer und die Anbindung an den Flugcontroller. Abschließend wird eine SIL-Testumgebung vorgestellt, mit der die Anzeige ohne Flugbetrieb geprüft werden kann.

4.2.1 Konzept und Zustandsdefinition

4.2.1.1 Anforderungen aus dem Regelwerk

Die zentrale Anforderung an die Anzeige stammt aus dem Regelwerk der ERC. REQ-FUN-070 fordert:

> „The rover and the drone shall emit visible signals when performing autonomy operations that are different from the signals emitted when being controlled by the team.“

Gefordert ist damit ein unterscheidbares Signal zwischen autonomem Betrieb und Steuerung durch das Team. Eine bestimmte Farbe schreibt das Regelwerk nicht vor. Die verwendeten WS2812B-LEDs besitzen eine Farbtiefe von 24 bit und können damit etwa 16,7 Millionen Mischfarben darstellen [2]. Um eine klare Unterscheidung auch aus größerer Entfernung zu gewährleisten, wurde die Anzeige dennoch auf wenige, deutlich verschiedene Farben beschränkt. Für den Zustand der Drohne werden die bekannten Farben industrieller Maschinenampeln übernommen: Grün für einen sicheren, normalen Betrieb, Gelb für Zustände, in denen gleich etwas passiert oder jemand handeln muss, und Rot für gefährliche Zustände und Fehler [1]. Für die Autonomie blieb damit die Farbe Blau, die mit keinem dieser Zustände verwechselt werden kann. Da das Autonomiesignal gerade in kritischen Situationen gebraucht wird, muss es auch während eines Fehlers jederzeit ablesbar bleiben. Weitere Anforderungen ergänzen diese Vorgabe:

REQ-DES-150
: Die wichtigsten Systeme sollen redundant ausgeführt sein. Daraus folgt, dass jedes der vier Landebeine einen eigenen, unabhängigen LED-Streifen erhält.

REQ-OPS-090 und REQ-OPS-100
: Nach dem Einschalten des Indikators bzw. nach dem Befehl zum autonomen Betrieb müssen jeweils

5 s vergehen, bevor die Drohne aktiv wird. Die Einhaltung dieser Wartezeit ist Aufgabe der Zustandsmaschine. Die Anzeige muss lediglich bereits leuchten, sobald der autonome Betrieb kommandiert ist. Zusätzlich macht der Statuskanal dieses Wartefenster sichtbar: Im Zustand `ARMED`, in dem die Motoren scharf geschaltet sind und die Drohne noch am Boden steht, leuchtet er gelb. Blinkt gleichzeitig der Autonomiekanal blau, steht ein autonomer Start unmittelbar bevor.

REQ-ENV-040
Die Drohne muss auch bei Kunstlicht oder Dunkelheit betrieben werden können. Selbstleuchtende LEDs erfüllen diese Anforderung grundsätzlich.

REQ-DES-060
Die Indikatorlampe soll bei Tageslicht aus 10 m Entfernung erkennbar sein. Formal gilt diese Anforderung nur für den Rover, sie wird für die Drohne aber als Designziel übernommen.

4.2.1.2 Zwei-Kanal-Modell

Ein erster Entwurf ordnete jedem Zustand genau eine Farbe zu, ein Fehler hätte beispielsweise alle LEDs rot gefärbt. Damit würde das Autonomiesignal genau dann verschwinden, wenn es am wichtigsten ist, nämlich bei einem Notfall im autonomen Flug. Dies widerspricht REQ-FUN-070. Die Anzeige wurde deshalb in zwei unabhängige Kanäle aufgeteilt, die auf getrennten LEDs dargestellt werden und sich gegenseitig nicht überdecken können:

Autonomiekanal
Zeigt an, ob die Drohne autonom fliegt oder vom Team gesteuert wird.

Statuskanal
Zeigt die aktuelle Flugphase und mögliche Fehler in den Farben Grün, Gelb und Rot an.

Jeder der vier Streifen besteht aus fünf LEDs, deren Belegung Abbildung 4.2 zeigt. LED 0 dient als Navigationslicht und leuchtet dauerhaft, am linken Bein grün und am rechten Bein rot. LED 1 und LED 2 bilden den Autonomiekanal, LED 3 und LED 4 den Statuskanal. Da beide Kanäle auf jedem Bein vorhanden sind, kostet der Ausfall eines Streifens zwar Helligkeit, aber keine Information. Einzige Ausnahme ist das Lauflicht im Zustand `READY`, das bewusst über die Beine wandert (siehe Abschnitt 4.2.1.3).

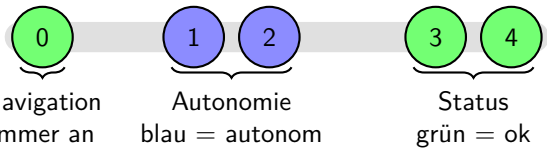


Abbildung 4.2: Belegung eines LED-Streifens am Beispiel eines linken Beins im autonomen Flug ohne Fehler. LED 0 zeigt die Seite an (links grün, rechts rot), LED 1–2 den Autonomiekanal, LED 3–4 den Statuskanal.

Der Nutzen der Trennung zeigt sich an zwei Situationen, die bei einer einfachen Farbzuordnung gleich aussehen würden. Bei einer Notlandung im autonomen Flug blinkt der Statuskanal rot, während der Autonomiekanal weiterhin blau blinkt. Bei einem Systemfehler, während der Pilot fliegt, blinkt der Statuskanal ebenfalls rot, der Autonomiekanal bleibt jedoch aus. Beobachter können also auch im Fehlerfall erkennen, wer die Drohne steuert.

4.2.1.3 Zuordnung von Zuständen zu Anzeigemustern

Die beiden Kanäle werden aus unterschiedlichen Informationen abgeleitet. Der Autonomiekanal richtet sich nach dem Flugmodus des Flugcontrollers, auf dem die Software PX4 läuft, und nicht nach der Flugphase

der Zustandsmaschine. Der Flugmodus beantwortet direkt die Frage, wer den Regelkreis schließt. In den Modi `OFFBOARD` und `AUTO.*` fliegen der Companion Computer bzw. der Autopilot, und der Kanal blinkt blau. In Modi wie `MANUAL` oder `POSCTL` befindet sich der Pilot in der Regelschleife, und der Kanal bleibt aus. Eine Flugphase wie `HOVER` sagt darüber nichts aus. Alle 14 Flugmodi sind ausdrücklich einer der beiden Klassen zugeordnet. Kommen Daten an, ist der Modus aber keiner Klasse zugeordnet, blinkt der Kanal schnell rot.

Der Statuskanal wird aus der Flugphase der Zustandsmaschine und zwei Fehlerfeldern abgeleitet. Dabei gilt eine feste Prüfreihenfolge, in der die erste zutreffende Bedingung entscheidet:

Keine Verbindung
: Besteht keine Verbindung zum Flugcontroller oder meldet die Zustandsmaschine den Systemfehler `FC_ERROR`, blitzt der ganze Streifen (siehe Abschnitt 4.2.1.4).

Fehler
: Ein anderer Systemfehler oder der Missionsfehler `EMERGENCY` führt zu schnellem rotem Blinken.

Unterbrechung
: Die Missionsfehler `ABORT` und `NAVIGATION_ERROR` führen zu gelbem Blinken mit 1 Hz.

Flugphase
: Andernfalls entscheidet die Flugphase nach Tabelle 4.3.

Die 13 Flugphasen der Zustandsmaschine werden auf acht Muster abgebildet. Zwei Zustände teilen sich ein Muster, wenn der Beobachter gleich reagieren soll oder der Kontext sie ohnehin unterscheidet. So sind `HOVER` und `APPROACHING` beide normaler Flug, ob die Drohne steht oder ein Ziel anfliegt, ist an der Drohne selbst zu sehen. `STOP` und `LOST` bedeuten beide, dass die Mission unterbrochen ist, aber noch keine Gefahr besteht und jemand handeln muss. `EMERGENCY_LANDING` und die beiden Systemfehlerzustände verlangen dieselbe Reaktion, nämlich Abstand zu halten. Die Übergänge zwischen Boden und Flug erhalten eigene Muster: `TAKEOFF` wechselt zwischen dem Gelb von `ARMED` und dem Grün des Flugs, `LANDING` blinkt grün.

Für den Zustand `READY` wurde ein grünes Lauflicht gewählt. Ein dunkler Statuskanal würde genauso aussehen wie im Ruhezustand `REST`. Das Lauflicht zeigt dagegen, dass das System läuft und bereit, aber nicht scharf geschaltet ist, ohne mit einem Flugzustand verwechselt werden zu können. Es läuft von oben gesehen im Uhrzeigersinn über die Beine, von vorne links über vorne rechts und hinten rechts nach hinten links, mit 0,25 s pro Bein. Der Missionsfehler `ABORT` wird direkt ausgewertet, da die Zustandsmaschine ihn etwa 100 ms vor dem Wechsel nach `STOP` setzt. Die Anzeige bleibt so über diesen Zwischenschritt stabil gelb. Ein unbekannter Zustand führt bewusst zu Rot und nicht zu einer dunklen Anzeige, da diese nicht von einer fehlerfreien, unbewaffneten Drohne zu unterscheiden wäre.

Tabelle 4.4 zeigt das Zeitverhalten der einzelnen Muster. Innerhalb einer Farbe unterscheiden sich die Muster durch ihre Bewegung, also statisch, blinkend, im Wechsel oder als Lauflicht. Blau blinkt mit 1 Hz, damit das Blinken auch aus der Entfernung klar erkennbar ist. Rot blinkt mit etwa 3,3 Hz deutlich schneller und wirkt dadurch alarmierend. Gelb wird nicht als gleiche Mischung aus Rot und Grün erzeugt, sondern mit einem geringeren Grünanteil, da der grüne Chip der WS2812B deutlich heller leuchtet als der rote. Bei gleichen Anteilen würde Gelb gelbgrün wirken und wäre von den grünen Mustern schwer zu unterscheiden.

4.2.1.4 Verhalten bei fehlenden oder ungültigen Daten

Für fehlende oder ungültige Daten gilt der Grundsatz, dass die Anzeige nie etwas behaupten darf, das sie nicht weiß. Ist der Flugmodus unbekannt, obwohl Daten ankommen, blinkt der Autonomiekanal deshalb schnell rot. Blau würde in diesem Fall einen autonomen Betrieb behaupten, der nicht bestätigt ist. Eine dunkle Anzeige würde dagegen behaupten, dass ein Mensch fliegt, und wäre damit die gefährlichere Fehlaussage.

Kommen gar keine gültigen Daten an, blitzt der ganze Streifen einschließlich des Navigationslichts mit

Tabelle 4.3: Zuordnung von Drohnenzustand zu Anzeigemuster je Kanal

Kanal	**Bedingung**	**Muster**
Autonomie (LED 1–2)	PX4-Modus `OFFBOARD` oder `AUTO.*`	blau, 1 Hz
	PX4-Modus mit Pilot (`MANUAL`, `POSCTL`, ...)	aus
	Modus unbekannt	rot, ca. 3,3 Hz
Status (LED 3–4)	`REST`, `EXIT`	aus
	`READY`	grünes Lauflicht
	`ARMED`	gelb, statisch
	`TAKEOFF`	grün/gelb im Wechsel
	`LANDING`	grün, 1 Hz
	`HOVER`, `APPROACHING`	grün, statisch
	`STOP`, `LOST`, Missionsfehler `ABORT`	gelb, 1 Hz
	`EMERGENCY_LANDING`, Systemfehler, Missionsfehler `EMERGENCY`	rot, ca. 3,3 Hz
Navigation (LED 0)	immer	links grün, rechts rot
ganzer Streifen	keine Daten, keine Verbindung zum Flugcontroller	rot, alle LEDs, 5 Hz

Tabelle 4.4: Zeitverhalten der Anzeigemuster

Muster	**Farbe(n)**	**Zeitverhalten**
statisch	grün oder gelb	dauernd an
blinkend	grün, gelb oder blau	1 Hz (0,5 s an, 0,5 s aus)
Wechsel	grün und gelb	je 0,5 s, ohne Dunkelphase
Lauflicht	grün	0,25 s pro Bein, eine Runde pro Sekunde
schnell blinkend	rot	ca. 3,3 Hz (0,15 s an, 0,15 s aus)
Streifen blitzt	rot, alle fünf LEDs	5 Hz (0,1 s an, 0,1 s aus)

5 Hz rot. Für diesen Fall wurde bewusst ein eigenes Signal gewählt, da sich „Fehler“ und „keine Daten“ inhaltlich unterscheiden. Bei einem Fehler weiß die Drohne, dass etwas nicht stimmt, und meldet dies. Die Navigationslichter leuchten dann ruhig, und nur der Statuskanal blinkt rot. Bei fehlenden Daten ist dagegen nichts über den Zustand bekannt. Das Blitzen des gesamten Streifens kann von keinem Zustand erzeugt werden, den die Drohne tatsächlich meldet, und ist damit eindeutig. Tabelle 4.5 zeigt die Reaktion der Anzeige in den einzelnen Fällen.

Tabelle 4.5: Verhalten der Anzeige bei fehlenden oder ungültigen Daten

Situation	**Reaktion**
Nach dem Start, noch keine gültige Nachricht	ganzer Streifen blitzt
Einzelne fehlerhafte Nachricht	ignorieren, letztes Muster halten
1 s lang keine gültige Nachricht (10 verpasste bei 10 Hz)	ganzer Streifen blitzt
Keine Verbindung zum Flugcontroller (`connected = False`)	ganzer Streifen blitzt
Daten des Flugcontrollers veraltet (`sys_error = FC_ERROR`)	ganzer Streifen blitzt
Beenden des LED-Knotens	alle LEDs aus

Ob die Zustandsmaschine ausgefallen ist oder nur fehlerhafte Nachrichten ankommen, kann die Anzeige nicht unterscheiden. Beide Fälle werden deshalb durch denselben Timeout von 1 s abgedeckt. Der Systemfehler `FC_ERROR` wird wie eine fehlende Verbindung behandelt. Die Zustandsmaschine setzt ihn, wenn die Daten des Flugcontrollers älter als 1,2 s sind, keine Verbindung besteht oder PX4 sich noch in der Initialisierung befindet. In allen drei Fällen ist der gemeldete Flugmodus nicht mehr verlässlich. Andere Systemfehler lassen den Autonomiekanal dagegen unverändert, da die Verbindung zum Flugcontroller dann noch besteht und der Modus weiterhin stimmt.

4.2.2 Hardware

4.2.2.1 LED-Streifen und Anordnung an den Landebeinen

Die Anzeige besteht aus 20 adressierbaren RGB-LEDs vom Typ WS2812B, aufgeteilt auf vier Streifen mit je fünf LEDs, also einen Streifen pro Landebein. Die Streifen werden in den dafür vorgesehenen Kanälen auf der Innenseite der Motorfüße befestigt (siehe Tabelle 4.1). Jede LED erwartet drei Byte Farbdaten in der Reihenfolge Grün, Rot, Blau. Farbreihenfolge und Pinbelegung wurden mit einem eigenen Inbetriebnahmewerkzeug (`led_selftest`) an der Drohne bestätigt. Tabelle 4.6 zeigt die Zuordnung der Beine zu den General Purpose Input/Output (GPIO)-Pins des Raspberry Pi.

Tabelle 4.6: Zuordnung der Landebeine zu den GPIO-Pins (BCM-Nummerierung)

Bein	**GPIO**	**Navigationslicht**
vorne links	22	grün
hinten links	27	grün
vorne rechts	23	rot
hinten rechts	24	rot

Der Raspberry Pi gibt seine Datensignale mit 3,3 V aus. Das liegt unter der Schaltschwelle, die das Datenblatt der LEDs bei einer Versorgung mit 5 V angibt [2]. Ein Pegelwandler vom Typ 74AHCT125 mit einem Gatter pro Bein ist deshalb vorgesehen, zum Zeitpunkt dieser Arbeit aber noch nicht verbaut (siehe Abschnitt 6.2).

4.2.2.2 Ansteuerung über den PIO-Block des RP1

Die Ansteuerung der LEDs ist auf dem Raspberry Pi 5 nicht auf dem naheliegenden Weg möglich. Der Raspberry Pi 5 verwendet einen neuen Ein- und Ausgabe-Chip, den RP1. Die verbreitete Bibliothek `rpi_ws281x` erzeugt das zeitkritische Signal über das DMA- und PWM-Timing der älteren Modelle und funktioniert deshalb auf dem Raspberry Pi 5 nicht. SPI-basierte Treiber sind zwar verfügbar, bedienen aber nur einen einzelnen Streifen am MOSI-Pin. Vier Streifen ließen sich damit nur hintereinandergeschaltet betreiben, was der angestrebten Fehlerisolation widerspricht.

Gewählt wurde deshalb der Programmable Input/Output (PIO)-Block des RP1, angesprochen über die Bibliothek `Adafruit-Blinka-Raspberry-Pi5-Neopixel`. Der PIO-Block kann jeden GPIO-Pin bedienen und erzeugt das zeitkritische Bitmuster in Hardware. Die Bibliothek nutzt dabei nur eine PIO-State-Machine und schaltet diese bei jedem Aufruf auf den jeweiligen Pin um. Die vier Beine werden also nacheinander beschrieben, wofür insgesamt etwa 3 ms bis 4 ms benötigt werden. Bei einer Renderperiode von 50 ms ist das unkritisch, und der Versatz zwischen den Beinen ist mit bloßem Auge nicht sichtbar.

Voraussetzung für diesen Weg ist, dass der Kernel den Treiber `rp1-pio` enthält, der das Gerät `/dev/pio0` bereitstellt. Wie in Abschnitt 4.2.5 beschrieben, hat diese Voraussetzung direkte Folgen für den Testaufbau.

4.2.2.3 Fehlerisolation durch getrennte Datenleitungen

Jeder Streifen ist an einen eigenen GPIO-Pin angeschlossen, die Streifen sind also nicht hintereinandergeschaltet. Eine fehlerhafte Lötstelle oder ein abgerissenes Kabel an einem Bein kann dadurch die übrigen drei Beine nicht beeinträchtigen. Die Software setzt dieses Prinzip fort, indem jedes Bein ein eigenes Treiberobjekt besitzt und Schreibfehler für jedes Bein einzeln abgefangen werden (siehe Abschnitt 4.2.3).

4.2.3 Software: Zustandsautomat auf dem Companion Computer

Die Software der Anzeige ist selbst kein Zustandsautomat. Sie liest den Zustand der bestehenden Zustandsmaschine des Teams aus und bildet ihn auf Anzeigemuster ab.

4.2.3.1 Schichtenarchitektur

Die Software ist als Robot Operating System (ROS) 2-Paket (`icarus_led_controller`) in Python umgesetzt und läuft unter ROS 2 Jazzy. Sie ist in drei Schichten gegliedert:

ROS-Schicht
: Der LED-Knoten (`led_node`) empfängt die Zustandsnachrichten, steuert die Zeitabläufe und legt fest, wie auf Fehler reagiert wird. Diese Schicht ist bewusst dünn gehalten.

Logikschicht
: Sie dekodiert die Nachrichten, ordnet ihnen Anzeigemuster zu und berechnet die Farben jedes Beins. Die Logikschicht ist reines Python und kommt ohne ROS und ohne Hardware aus.

Hardwareschicht
: Eine Treiberschnittstelle (`LedStripDriver`) mit genau einer Methode, `set_pixels()`. Dahinter steht entweder der echte Treiber für den PIO-Block oder ein Platzhalter, der die Farben nur protokolliert.

Die Trennung verfolgt zwei Ziele. Zum einen lässt sich die gesamte Logik ohne Drohne, ohne Raspberry Pi und ohne ROS testen. Zum anderen kann der Hardwaretreiber ausgetauscht werden, etwa gegen eine SPI-Lösung oder einen Mikrocontroller, ohne dass sich an der Logik etwas ändert. Beim Start prüft der

Knoten, ob die PIO-Bibliothek verfügbar ist, und wählt danach den echten Treiber oder den Platzhalter. Dadurch läuft dasselbe Paket unverändert auf einem Laptop und auf dem Raspberry Pi.

4.2.3.2 Auslesen des Zustands-Topics

Die Zustandsmaschine des Teams veröffentlicht ihren Zustand alle 100 ms, also mit 10 Hz, auf dem Topic `statemachine/state`. Dabei wird jeweils das komplette Zustandsobjekt mit 61 Feldern übertragen, darunter Flugphase, Fehler, Flugmodus, Position, Batterie und Missionsdaten. Das Objekt wird im binären Format Concise Binary Object Representation (CBOR) serialisiert, einem JSON-ähnlichen Format nach RFC 8949. Da das Topic den Nachrichtentyp `std_msgs/String` besitzt, werden die Bytes als Hexadezimaltext übertragen. Eine Nachricht ist etwa 2,2 kB groß. Aufzählungstypen (Enums) aus Python kennt CBOR nicht, sie werden deshalb als markierter Wert aus Name und Klassenname geschrieben.

Der LED-Knoten verarbeitet eine Nachricht in drei Schritten. Zuerst wandelt er den Hexadezimaltext zurück in Bytes. Anschließend dekodiert er die CBOR-Daten und baut dabei die Aufzählungstypen wieder auf. Ein unbekannter Aufzählungstyp wird dabei zu einfachem Text, statt die Dekodierung abzubrechen, sodass ein neuer Typ in einem Feld, das die Anzeige nicht benötigt, nicht stört. Zuletzt werden nur die fünf Felder übernommen, die die Anzeige benötigt, und streng auf ihren Typ geprüft (Tabelle 4.7).

Tabelle 4.7: Vom LED-Knoten ausgewertete Felder des Zustandsobjekts

Feld	Typ	Verwendung
`state`	`SM_States`	Statuskanal (Flugphase)
`mission_error`	`SM_Mission_Errors`	Statuskanal
`sys_error`	`SM_System_Errors`	Statuskanal, `FC_ERROR` auch für die Verbindung
`current_mode`	`FC_Modes`	Autonomiekanal
`connected`	`bool`	gibt an, ob `current_mode` gültig ist

Das Feld `connected` ist notwendig, weil das Zustandsobjekt für den Flugmodus den Standardwert `AUTO.LOITER` besitzt. Ohne Verbindung zum Flugcontroller stünde dort also ein autonomer Modus, obwohl nie ein Modus gemeldet wurde, und eine Drohne mit abgestecktem Flugcontroller würde blau blinken. Die strenge Typprüfung verhindert zudem, dass ein zu Text gewordener unbekannter Wert als gültiger Zustand durchrutscht. Jeder Fehler beim Dekodieren führt zu einer einzigen Ausnahme. Wie die Anzeige darauf reagiert, entscheidet der Knoten und nicht der Decoder. Das Auslesen der Daten und das Anzeigeverhalten sind damit voneinander getrennt.

Das Nachrichtenformat wurde bewusst nicht verändert, da die Bodenstationsoberfläche des Teams denselben Datenstrom liest und durch eine Formatänderung beeinträchtigt werden könnte. Der zusätzliche Aufwand für die Dekodierung ist vernachlässigbar: Auf einem x86-Rechner benötigt sie etwa 0,05 ms pro Nachricht.

4.2.3.3 Zeitsteuerung und Blinkmuster

Der LED-Knoten arbeitet mit zwei voneinander unabhängigen Abläufen. Beim Empfang einer Nachricht wird diese dekodiert, auf zwei Anzeigemuster abgebildet und zusammen mit der Empfangszeit gespeichert, ohne dass dabei LEDs beschrieben werden. Ein eigener Timer zeichnet die Anzeige mit 20 Hz. Er prüft zunächst, ob die Daten veraltet sind (siehe Abschnitt 4.2.1.4), und beschreibt dann alle vier Beine. Die Trennung ist notwendig, weil die Blinkmuster auch dann weiterlaufen müssen, wenn keine Nachricht eintrifft, und die Anzeige gerade in diesem Fall auf „keine Daten“ wechseln soll. Die Frequenz von 20 Hz ergibt sich aus der kürzesten Phase: Beim Blitzen des Streifens dauert eine Phase 0,1 s, sodass zwei Zeichenvorgänge

in jede Phase fallen, beim roten Fehlerblinken sind es drei. Da ein Durchgang für alle vier Beine nur etwa 4 ms dauert, könnte die Rate bei Bedarf ohne Risiko erhöht werden.

Jedes Muster ist als kurze Folge von Farben $F_0, \ldots, F_{n-1}$ hinterlegt, die mit einem festen Takt Δt durchlaufen wird. Ein blaues Blinken besteht zum Beispiel aus der Folge (blau, aus) mit $\Delta t = 0{,}5\,\text{s}$, der Wechsel bei `TAKEOFF` aus (grün, gelb). Welche Farbe eine LED gerade zeigt, wird nicht über einen internen Takt bestimmt, sondern als reine Funktion von Muster, aktueller Zeit t und Bein berechnet:

$$k = \left\lfloor \frac{t}{\Delta t} \right\rfloor - s, \qquad \text{Farbe} = F_{k \bmod n} \tag{4.1}$$

Dabei ist s die Position des Beins in der Reihenfolge des Lauflichts. Für alle anderen Muster gilt $s = 0$. Da alle vier Beine denselben Zeitstempel erhalten, blinken sie garantiert synchron. Zudem lässt sich das Verhalten mit vorgegebenen Zeitstempeln testen. Das Lauflicht entsteht aus derselben Formel: Seine Farbfolge (grün, aus, aus, aus) hat genau einen grünen Schritt, und jedes Bein ist um seine Position s verschoben. Zu jedem Zeitpunkt leuchtet deshalb genau ein Bein grün, und das grüne Bein wandert weiter. Als Kompromiss wird in Kauf genommen, dass die Phase frei gegenüber der absoluten Zeit läuft und nicht mit dem Musterwechsel beginnt. Ein Wechsel auf ein blinkendes Muster kann deshalb mit einer bis zu 0,5 s langen Dunkelphase beginnen.

4.2.3.4 Robustheit im Betrieb

Fehlerisolation pro Bein
Tritt beim Beschreiben eines Beins ein Fehler auf, wird dieser abgefangen und nur einmal beim Übergang von funktionsfähig zu defekt protokolliert. Die übrigen drei Beine werden weiter angesteuert. Ohne diese Maßnahme hätte ein einzelner Schreibfehler den Knoten beendet, und alle vier Beine wären dunkel geblieben.

Ausschalten beim Beenden
WS2812B-LEDs halten ihr letztes Bild unbegrenzt. Ein gestoppter Knoten würde deshalb zum Beispiel „autonom, fehlerfrei“ stehen lassen. Beim Beenden des Knotens werden daher alle LEDs ausgeschaltet. Dies greift allerdings nur bei einem geordneten Beenden. Gegen einen Absturz des Knotens hilft nur ein Watchdog, der bisher nicht umgesetzt ist.

Begrenzte Protokollierung
Wiederkehrende Warnungen, etwa zu fehlerhaften Nachrichten oder veralteten Daten, werden höchstens einmal pro Sekunde ausgegeben.

4.2.4 Anbindung an den Flugcontroller

4.2.4.1 Datenpfad vom Flugcontroller zum LED-Knoten

Der LED-Knoten kommuniziert nicht direkt mit dem Flugcontroller, sondern ist ausschließlich an die Zustandsmaschine des Teams angebunden. Damit gibt es nur eine Stelle, an der der Zustand der Drohne bewertet wird, und die Anzeige zeigt genau den Zustand, von dem auch Zustandsmaschine und Bodenstation ausgehen. Der Datenpfad verläuft in folgenden Schritten:

Flugcontroller
Der Flugcontroller, ein Pixhawk 6C mit PX4, sendet seine Daten über Micro Air Vehicle Link (MAVLink) per Universal Asynchronous Receiver Transmitter (UART) an den Raspberry Pi.

MAVROS
MAVROS, die Brücke zwischen MAVLink und ROS, veröffentlicht unter anderem Verbindungsstatus

und Flugmodus auf dem Topic `/mavros/state`.

Zustandsmaschine
Die Zustandsmaschine übernimmt diese Werte, ergänzt ihre eigene Flugphase sowie die Fehlerauswertung und veröffentlicht das Ergebnis mit 10 Hz auf `statemachine/state`.

Bodenstation und LED-Knoten
Diesen Datenstrom lesen sowohl die Bodenstationsoberfläche als auch der LED-Knoten, der daraus die Anzeige der vier Streifen berechnet.

4.2.4.2 Bestimmung des Autonomiezustands aus dem PX4-Flugmodus

MAVROS meldet den PX4-Flugmodus als Text, zum Beispiel `POSCTL` oder `AUTO.LOITER`. Die Zustandsmaschine wandelt diesen Text in einen Aufzählungstyp mit 14 Werten um, den der LED-Knoten anschließend wie in Abschnitt 4.2.1.3 beschrieben klassifiziert. Auch das Feld `connected` stammt aus `/mavros/state`.

Die Werte `connected` und `current_mode` werden allerdings nur aktualisiert, wenn eine Nachricht von MAVROS ankommt. Stürzt MAVROS ab, sendet die Zustandsmaschine weiterhin mit 10 Hz, während `connected` auf `True` und der Flugmodus auf dem letzten Wert stehen bleiben. Der Timeout des LED-Knotens greift in diesem Fall nicht, da weiterhin gültige Nachrichten ankommen. In einer früheren Fassung zeigte der Autonomiekanal deshalb weiter den alten Modus an, nach `MANUAL` also eine dunkle Anzeige, obwohl nicht mehr bekannt war, wer die Drohne steuert. Gelöst wurde dies, indem der LED-Knoten den Systemfehler `FC_ERROR` wie eine fehlende Verbindung behandelt. Die Zustandsmaschine setzt diesen Fehler nach etwa 1,2 s bis 1,7 s, danach blitzt der ganze Streifen. Die Lösung kommt ohne Änderung an der Zustandsmaschine oder am Datenformat aus. Eine weitere bekannte Lücke im Datenpfad liegt außerhalb der LED-Software und wird in Abschnitt 6.3.2 behandelt.

4.2.5 Software-in-the-Loop-Testumgebung

4.2.5.1 Motivation

Der Software-Stack für den Flug, bestehend aus Ubuntu 24.04, ROS 2 Jazzy und MAVROS, läuft auf einem Ubuntu-Kernel, der den Treiber `rp1-pio` nicht enthält. Ohne das Gerät `/dev/pio0` lassen sich die LEDs dort nicht ansteuern. Auf einer zweiten SD-Karte mit Raspberry Pi OS Lite steht `/dev/pio0` zur Verfügung, dort fehlt jedoch der Flug-Stack. Die Anzeige sollte trotzdem mit echter Hardware und echtem Nachrichtenformat geprüft werden, und zwar für alle Zustandskombinationen, auch für gefährliche wie eine Notlandung, die man im Flug nicht absichtlich herbeiführt.

Die Lösung ist eine SIL-Testumgebung. LED-Knoten, Decoder, Zuordnung, Treiber und LEDs sind dabei echt, lediglich die Zustandsmaschine wird durch einen simulierten Zustandsgeber ersetzt. Gesteuert wird dieser über die Kommandozeile direkt auf dem Raspberry Pi, ohne zusätzliche Software auf einem Laptop. Im Team wird der Begriff SIL auch für die Simulation des gesamten Flugs mit Gazebo und PX4 verwendet. Hier ist dagegen eine Testumgebung für die Anzeige gemeint, bei der Anzeigesoftware und LEDs echt sind und nur die Quelle des Zustands simuliert wird.

4.2.5.2 Aufbau

Abbildung 4.3 zeigt den Aufbau der Testumgebung. Auf dem Raspberry Pi läuft Raspberry Pi OS Lite, ROS 2 Jazzy wird in einem Docker-Container auf Basis des Images `ros:jazzy` ausgeführt. Der Container ist notwendig, weil ROS 2 Jazzy offiziell für Ubuntu 24.04 gepflegt wird und nicht für das auf Debian basierende Raspberry Pi OS. Der Container stellt die Ubuntu-Umgebung bereit, während der Kernel des Raspberry Pi OS das Gerät `/dev/pio0` liefert.

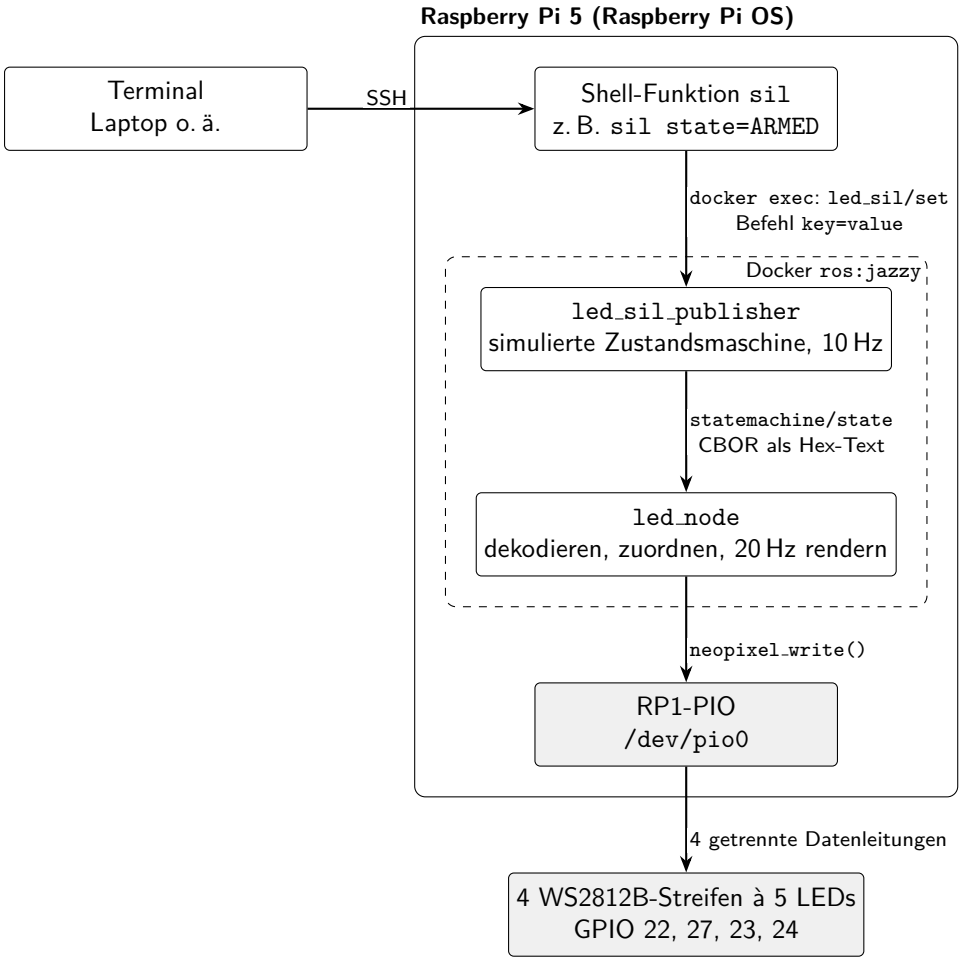


Abbildung 4.3: Aufbau der Software-in-the-Loop-Testumgebung der LED-Statusanzeige

Der ROS-Workspace liegt auf dem Raspberry Pi und ist in den Container eingebunden. Er enthält das LED-Paket, das gemeinsame Paket mit den Aufzählungstypen und das Paket der Zustandsmaschine, das nur wegen seines Encoders benötigt wird. Neuer Code gelangt so per `git pull` und anschließendem Bauen im Container auf den Raspberry Pi, ohne dass das Image neu erstellt werden muss. Hauptprozess des Containers ist eine Launch-Datei, die LED-Knoten und simulierten Zustandsgeber gemeinsam startet. Weder MAVROS noch die echte Zustandsmaschine werden dabei gestartet, sodass der Testaufbau den Flugcontroller grundsätzlich nicht ansteuern kann. Bedient wird die Testumgebung per SSH auf dem Raspberry Pi. Da keine ROS-Kommunikation über das WLAN läuft, spielen Netzwerkeinstellungen von ROS für den Test keine Rolle.

4.2.5.3 Simulierter Zustandsgeber

Der simulierte Zustandsgeber verwaltet genau die fünf Felder, die die Anzeige auswertet, sowie einen Schalter, mit dem das Senden ein- und ausgeschaltet wird. Er startet im Zustand `REST` im Flugmodus `POSCTL`, mit bestehender Verbindung und ohne Fehler, sodass zu Beginn nur die Navigationslichter leuchten. Rot oder ein blitzender Streifen bedeuten in der Testumgebung damit immer, dass dieser Zustand ausgewählt wurde oder keine Daten ankommen, und nie einen Standardwert.

Befehle erhält der Zustandsgeber als Textzeile mit Schlüssel-Wert-Paaren auf dem Topic `led_sil/set`, zum Beispiel `state=HOVER mode=AUTO.LOITER`. Nicht angegebene Werte bleiben erhalten. Enthält ein Befehl einen ungültigen Teil, wird er vollständig verworfen, da ein nur teilweise angewendeter Befehl eine Kombination anzeigen würde, die niemand ausgewählt hat.

Aus diesen Werten erzeugt der Zustandsgeber ein echtes Zustandsobjekt und veröffentlicht es wie die echte Zustandsmaschine mit 10 Hz auf `statemachine/state`. Als Name trägt das Objekt `LED_SIL`, damit Aufzeichnungen und Bodenstation simulierte Daten erkennen. Die wichtigste Designentscheidung ist, dass der Zustandsgeber den Encoder der echten Zustandsmaschine importiert, statt ihn zu kopieren. Eine Kopie könnte unbemerkt vom echten Format abweichen, und der Test würde weiterhin bestehen, ohne noch etwas zu beweisen. So durchläuft der LED-Knoten in der Testumgebung exakt denselben Codepfad wie im Flug. Zwei Schutzmechanismen ergänzen den Zustandsgeber:

Zweiter Publisher
: Erscheint ein weiterer Publisher auf `statemachine/state`, wahrscheinlich die echte Zustandsmaschine, verstummt der Zustandsgeber und meldet einen Fehler. Andernfalls würden sich echte und simulierte Daten mischen, und Anzeige sowie Bodenstation würden einen Zustand zeigen, in dem die Drohne nicht ist. Dies funktioniert unabhängig von der Startreihenfolge.

Simulierter Ausfall
: Über den Schalter kann das Senden unterbrochen werden, um eine ausgefallene Zustandsmaschine nachzubilden. Der LED-Knoten muss diesen Fall selbst erkennen, nach 1 s blitzt der ganze Streifen.

4.2.5.4 Bedienung über die Kommandozeile

Ein Test soll ohne zusätzliche Software auf dem Laptop möglich sein. Dazu meldet man sich per SSH am Raspberry Pi an und gibt zum Beispiel `sil state=ARMED mode=OFFBOARD` ein. Die LEDs reagieren sofort, und man kann direkt neben der Drohne stehen. Der Befehl `sil` ist eine Shell-Funktion, die auf dem Raspberry Pi selbst und nicht im Container hinterlegt ist. Sie bleibt dadurch auch erhalten, wenn der Container neu erstellt wird. Die Funktion arbeitet in drei Schritten:

Container finden
: Der laufende Container wird über sein Image gesucht und nicht über einen Namen. Das funktioniert auch nach einem Neuerstellen mit neuer Kennung.

Befehl übergeben
: Über `docker exec` wird der Befehl im Container einmalig auf dem Topic `led_sil/set` veröffentlicht. Läuft der Zustandsgeber nicht, bricht die Funktion nach 10 s mit einer Meldung ab.

Rückmeldung zeigen
: Die Funktion gibt die letzte Meldung des Zustandsgebers aus. Ein Tippfehler wird dort mit der Liste der erlaubten Werte abgelehnt, und der vorherige Zustand bleibt erhalten.

Mit `sil reset` wird der Startzustand wiederhergestellt. Jeder Befehl ist damit reproduzierbar und kann direkt in ein Testprotokoll übernommen werden.

Ursprünglich wurde die Testumgebung über eine grafische Oberfläche auf dem Laptop bedient, die zusätzlich anzeigte, was die LEDs zeigen sollten. Diese benötigte jedoch ROS 2 auf dem Laptop, gleiche Netzwerkeinstellungen auf beiden Geräten und eine Freigabe in der Firewall. Außerdem sendete sie ihren Zustand jede Sekunde neu und überschrieb damit jeden anderen Befehl. Die Bedienung über die Kommandozeile hat sie deshalb ersetzt.

4.2.5.5 Aussagekraft und Grenzen

Die Testumgebung belegt, dass das echte Nachrichtenformat der Zustandsmaschine korrekt gelesen wird, dass Zuordnung und Blinkfrequenzen auf echter Hardware stimmen und dass die Anzeige bei ausbleibenden Daten und bei `FC_ERROR` wie vorgesehen reagiert. Außerdem bestätigt sie Pinbelegung, Farbreihenfolge und PIO-Treiber an den echten Streifen sowie die Reihenfolge des Lauflichts passend zur tatsächlichen Lage der Beine.

Nicht belegt wird dagegen, ob die echte Zustandsmaschine die getesteten Kombinationen im Flug tatsächlich so erzeugt. Der Zustandsgeber erlaubt auch unrealistische Kombinationen wie die Flugphase `HOVER` im Flugmodus `MANUAL`. Ebenfalls nicht abgedeckt ist das Zeitverhalten der echten Zustandsmaschine unter Last. Bei einem Absturz von MAVROS wird nur die Reaktion der Anzeige geprüft, da `FC_ERROR` in der Testumgebung direkt gesetzt wird. Dass die echte Zustandsmaschine diesen Fehler rechtzeitig setzt, ist nur aus ihrem Code abgeleitet. Die Bodenstationsoberfläche des Teams ist an der Testumgebung nicht beteiligt. Ein Test im Flugbetrieb ist derzeit nicht möglich, da kein Betriebssystem beides bietet: Raspberry Pi OS unterstützt den PIO-Block, besitzt aber keinen Flug-Stack, während Ubuntu den Flug-Stack, aber keinen PIO-Treiber bereitstellt.

5 Evaluation

In diesem Kapitel werden die Entwickelten Methoden Evalutiert und Bewertet. Dabei wird die Evalutation des neuen Frames im Detail besprochen, sowie die Evaluation der Integration der Statusanzeige.

5.1 Vorgehen und Bewertungskriterien

Nach der Konstruktion und dem 3D-Drucken der Bauteile, werden diese Bewertet anhand von verschiedenen Kriterien und Testmethoden. Dazu gehören Maßgenauigkeit, kein übermäßiges verziehen während des Drucks und der eingehaltenen TOleranz. Durch das natürliche Verhalten von Thermoplasten, sind Toleranzangaben großzügig gehalten um kein zu großen Ausschuss zu erzeugen.

5.2 Evaluation des Frames

5.2.1 Masse

Beim messen der Masse des neuen Frames können zwei Wege benutzt werden. Zum einen wird die komplette Drohne mitsamt Elektronik und Accessoires gewogen, zum anderen werden die Berechnungen der Slicersoftware benutzt. Eine im FRoST-Labor verfügbare Waage war für das Abmessen des Gewichtes der gesamten Drohne. Bei der Berechnung in der Slicersoftware werden jedoch nur die neuen 3D-gedruckten Teile mit den alten Karbonplattenteilen verglichen um so das Gewicht von Schrauben, Einschmelzgewinden, Kabeln und sonstiger Elektronik zu vernachlässigen. Zudem werden die Filamente ASA und das gesponsorte PA11-CF-SL verglichen um weitere Vorteile des speziellen Filaments zu evaluieren.

5.2.2 Maßhaltigkeit und Montage

Die Maßhaltigkeit der gedruckten Bauteile wurde durch eine visuelle Prüfung auf Warping, bzw. Verzug nach dem Druck evaluiert. Das Einschmelzen der Gewinde, und die Genauigkeit davon, wurde durch ein anschließendes Einschrauben von passenden Schrauben geprüft, sowie ein testweises Zusammensetzen von zwei Komponenten.

5.2.3 Mechanische Belastbarkeit

Die Evaluierung von 3D-gedruckten Teilen mittels FEM-Analysen gestaltet sich durch die Geometrie mit Wänden und Infill von sehr schwierig bis unmöglich. So können die fertigen Bauteile nicht in der Simulation sondern durch praktische Tests, eingebaut in der Drohne, getestet werden. Die Verbindungsarme von der baseplate zu den Motoren aus dem PA11-CF-SL Filament wurden in die Drohne, gemeinsam mit den TPU Füßen, eingebaut und mit Freifalltests aus definierten Höhen evaluiert. Dabei wurde auf die Landung geachtet, wie stark die Bauteile sich verbiegen und ob Schäden durch den Test entstanden sind.

5.2.4 Fertigungsaufwand

Die Bewertung des Fertigungsaufwands wurde anhand der Nachbearbeitung der Bauteile festgelegt. Es wurde ermittelt, wie einfach oder einfach die Bauteile mit Einschmelzgewinden nachbearbeitet werden konnten. Ein gutes Erreichen von den Bohrungen für die Einschmelzgewinde, sowie die passende Größe dieser Bohrung, bedeutete eine einfache Nachbearbeitung. Druckspezifische Extrateile am Bauteil wie Supports, Mauseohren oder Brims verschlechtern jedoch die Bewertung, da es ein teils Zeitintensive Nachbearbeitung verlangt um das fertige Bauteil zu bekommen.

5.2.5 Wartbarkeit

Die Wartbarkeit des neuen Frames ist durch die benötigte Zeit zum Austauschen der Bauteile bestimmt, sowie aber auch der nötige Aufwand Kabel oder andere Elektronik zu tauschen. USB-Anschlüsse für Pi und Flight Controller, sowie Kabelführung, -halterung und -position sind bestimmend wie gut der neue Frame wartbar ist.

5.2.6 Flugtests

Die Evalution der Stabilität der Drohne während des Fluges ist, im Unterschied zu der Visuelle Bewertung der Bauteile, durch die Python Bibliothek pyulog und der Website `Flight Review` umgesetzt worden. Die Flight Review Website bietet eine graphische Ausarbeitung der Log dateien an, während pyulog nur Werte der Logs zum verleichen Anzeigen kann. Eine Erweiterung der Flight Review Website ist die VS Code Erweiterung PX4 ULog Viewer. Dort können auch lokal über die Zeit geplottet die Werte der Motoren, der Sensoren und vieler weiterer angezeigt werden, während bei der Website die Logs hochgeladen werden müssen.

5.3 Evaluation der Statusanzeige

Die Statusanzeige wird in mehreren Stufen evaluiert, die sich darin unterscheiden, wie viel vom späteren Einsatz sie abbilden. Automatisierte Tests prüfen die Anzeigelogik ohne Hardware. Der Funktionstest in der SIL-Testumgebung aus Abschnitt 4.2.5 prüft dieselbe Logik an den echten LEDs der Drohne. Ein Sichtbarkeitstest soll zeigen, ob die Muster aus der geforderten Entfernung erkennbar und unterscheidbar sind. Ein Test im Flugbetrieb ist derzeit nicht möglich, die Gründe dafür werden in Abschnitt 5.3.4 erläutert. Tabelle 5.1 ordnet den Anforderungen aus Abschnitt 4.2.1.1 die jeweilige Prüfung und das Abnahmekriterium zu.

5.3.1 Automatisierte Tests der Anzeigelogik

Die Anzeigelogik wird mit dem Testframework pytest geprüft. Die Tests laufen ohne Raspberry Pi, ohne LEDs und ohne laufendes ROS 2. Möglich wird das durch die Schichtenarchitektur aus Abschnitt 4.2.3: Die Logikschicht ist reines Python, und statt des echten Treibers wird eine Schreibfunktion eingesetzt, die die gesendeten Daten nur aufzeichnet. Damit lässt sich sogar prüfen, welche Bytes der Treiber an die LEDs senden würde. Die Tests sind auf zwei Dateien verteilt:

`test_led_logic.py` (60 Testfunktionen)
: Geprüft werden die Anzeigemuster und ihr Zeitverhalten sowie die Zuordnung von Zuständen zu Mustern. Für die Zuordnung wird nachgewiesen, dass jeder der 14 Flugmodi und jede der 13 Flugphasen genau einer Klasse zugeordnet ist, und jede Zeile von Tabelle 4.3 wird einzeln getestet. Weitere Tests betreffen die Prüfreihenfolge der Fehler, den Decoder einschließlich fehlerhafter Nachrichten,

Tabelle 5.1: Zuordnung der Anforderungen an die Statusanzeige zu Prüfung und Abnahmekriterium

Anforderung	Prüfung	Abnahmekriterium
REQ-FUN-070	automatisierte Tests, SIL-Matrix Zeilen 4, 6, 7, 10, 11	Der Autonomiekanal blinkt genau in den autonomen Flugmodi blau, auch wenn gleichzeitig ein Fehler angezeigt wird.
REQ-DES-150	automatisierte Tests (Fehlerisolation je Bein)	Ein Schreibfehler an einem Bein beeinträchtigt die übrigen Beine nicht.
REQ-OPS-090, REQ-OPS-100	SIL-Matrix Zeile 4	Das Autonomiesignal leuchtet bereits, bevor sich die Drohne bewegt. Die Wartezeit selbst einzuhalten ist Aufgabe der Zustandsmaschine.
REQ-DES-060	Sichtbarkeitstest	Die Muster sind bei Tageslicht aus 10 m Entfernung erkennbar.
REQ-ENV-040	Sichtbarkeitstest	Die Muster sind bei Kunstlicht und Dunkelheit erkennbar und unterscheidbar.

die Fehlerisolation je Bein, das Ausschalten beim Beenden und das Datenformat des Treibers, also die Farbreihenfolge Grün, Rot, Blau und die Länge des Puffers.

`test_sil.py` (10 Testfunktionen)
Geprüft werden der Befehlsparser des simulierten Zustandsgebers und der vollständige Weg einer Nachricht durch den echten Encoder der Zustandsmaschine und zurück durch den Decoder des LED-Knotens.

Mit der Überarbeitung der Anzeigemuster kamen Tests für die neuen Muster hinzu. Sie prüfen die Laufrichtung des Lauflichts von vorne links über vorne rechts und hinten rechts nach hinten links, den Wechsel zwischen Grün und Gelb ohne Dunkelphase und das Signal für fehlende Daten. Dieses muss den ganzen Streifen erfassen und mit mindestens 4 Hz schneller blitzen als das rote Fehlerblinken. Außerdem wird geprüft, dass das Navigationslicht bei einem Fehler ruhig bleibt.

Viele Tests sind parametrisiert und laufen zum Beispiel einmal pro Zustand, die Zahl der ausgeführten Testfälle ist deshalb deutlich größer als die Zahl der Testfunktionen. Zwei Regressionstests sichern die beiden wichtigsten Designentscheidungen ab:

`test_error_cannot_hide_the_autonomy_indicator`
Ein Fehler darf das Autonomiesignal nicht überdecken. Dieser Test sichert das Zwei-Kanal-Modell aus Abschnitt 4.2.1.2 und damit REQ-FUN-070 ab.

`test_fc_error_means_no_link_even_with_connected_frozen_true`
Der Systemfehler `FC_ERROR` wird auch dann als fehlende Verbindung gewertet, wenn `connected` eingefroren auf `True` steht. Dieser Test bildet den Absturz von MAVROS aus Abschnitt 4.2.4 nach.

Die Ergebnisse der Testläufe sind in Abschnitt 6.2 zusammengefasst.

5.3.2 Funktionstest an der Hardware

Die automatisierten Tests belegen die Logik, aber nicht, dass die LEDs an der Drohne tatsächlich das berechnete Muster zeigen. Dafür wird die SIL-Testumgebung aus Abschnitt 4.2.5 verwendet. Vorab wurden die Pinbelegung und die Farbreihenfolge mit dem Inbetriebnahmewerkzeug `led_selftest` an der Drohne bestätigt. Vor jedem Testlauf wird der aktuelle Stand der Software auf den Raspberry Pi gebracht. Dazu wird der Code per `git pull` aktualisiert, im Container mit `colcon build` gebaut und der Container anschließend neu gestartet.

Der Test folgt der Testmatrix in Tabelle 5.2. Jede Zeile beginnt im Ausgangszustand, der mit `sil reset` hergestellt wird. Danach wird der Befehl der Zeile per SSH auf dem Raspberry Pi eingegeben, und an

der Drohne werden Autonomie- und Statuskanal mit dem erwarteten Muster verglichen. Da jeder Befehl reproduzierbar ist, kann ein Test jederzeit wiederholt werden, zum Beispiel nach einer Änderung an der Software.

Tabelle 5.2: Testmatrix für den Funktionstest der Statusanzeige in der SIL-Testumgebung

Nr.	Befehl	Autonomie erwartet (LED 1–2)	Status erwartet (LED 3–4)
1	`sil reset` (`REST`, `POSCTL`, verbunden)	aus	aus, nur Navigationslicht
2	`sil state=READY`	aus	grünes Lauflicht im Uhrzeigersinn
3	`sil state=ARMED`	aus	gelb, statisch
4	`sil state=ARMED mode=OFFBOARD`	blau, 1 Hz	gelb, statisch
5	`sil state=TAKEOFF`	aus	grün/gelb im Wechsel
6	`sil state=HOVER mode=AUTO.LOITER`	blau, 1 Hz	grün, statisch
7	`sil state=APPROACHING mode=OFFBOARD`	blau, 1 Hz	grün, statisch
8	`sil state=LANDING`	aus	grün, 1 Hz
9	`sil state=STOP mission_error=ABORT`	aus	gelb, 1 Hz
10	`sil state=EMERGENCY_LANDING mode=AUTO.LAND mission_error=EMERGENCY`	blau, 1 Hz	rot, ca. 3,3 Hz
11	`sil state=A_SYS_ERROR mode=MANUAL sys_error=TOPIC_ERROR`	aus	rot, ca. 3,3 Hz, Navigationslicht ruhig
12	`sil sys_error=FC_ERROR`	ganzer Streifen blitzt	
13	`sil connected=false`	ganzer Streifen blitzt	
14	`sil publishing=false`	ganzer Streifen blitzt nach höchstens 1 s	
15	Testumgebung beenden (`docker stop`)	alle LEDs aus, auch das Navigationslicht	

Die Zeilen der Matrix sind so gewählt, dass jede Zeile eine bestimmte Eigenschaft der Anzeige belegt. Die Zeilen 1 bis 9 decken die Muster des Statuskanals im Normalbetrieb ab, einschließlich der Übergänge zwischen Boden und Flug. Zeile 4 zeigt das Wartefenster aus REQ-OPS-090 und REQ-OPS-100: Die Drohne ist scharf geschaltet und im autonomen Modus, steht aber noch, und das Autonomiesignal ist bereits sichtbar. Die Zeilen 10 und 11 belegen REQ-FUN-070 direkt. Beide sind Notfälle mit demselben roten Statusmuster, die Autonomie bleibt aber unterscheidbar, da der Autonomiekanal bei der Notlandung im Modus `AUTO.LAND` blau blinkt und beim Systemfehler im Modus `MANUAL` aus bleibt. Der Vergleich der Zeilen 11 und 12 belegt die Unterscheidung zwischen „Fehler“ und „keine Daten“ aus Abschnitt 4.2.1.4. Die Zeilen 12 bis 14 prüfen die drei Wege, auf denen die Anzeige fehlende oder unzuverlässige Daten erkennt, also den Systemfehler `FC_ERROR`, die fehlende Verbindung zum Flugcontroller und den Ausfall der Zustandsmaschine. Zeile 15 prüft schließlich, dass die LEDs beim Beenden ausgeschaltet werden und kein veraltetes Bild stehen bleibt.

Die beobachteten Muster je Zeile werden in Abschnitt 6.2 dem erwarteten Verhalten gegenübergestellt.

5.3.3 Sichtbarkeit und Unterscheidbarkeit

Der Funktionstest zeigt, dass die richtigen Muster angezeigt werden, aber nicht, ob ein Beobachter sie im Einsatz auch erkennt. Dies soll ein Sichtbarkeitstest prüfen. Als Maßstab dient REQ-DES-060, nach der eine Indikatorlampe bei Tageslicht aus 10 m Entfernung erkennbar sein muss. Formal gilt diese Anforderung nur für den Rover, sie wird für die Drohne aber als Designziel übernommen. Ergänzend wird nach REQ-ENV-040 auch bei Kunstlicht und Dunkelheit geprüft.

Bewertet werden soll nicht nur, ob die LEDs leuchten, sondern ob sich die Muster voneinander unterscheiden lassen. Kritisch ist dabei vor allem Gelb. Gelb ist ein Mischton aus Rot und Grün, und sein Eindruck hängt von den LEDs und der Helligkeit ab. Ob Gelb bei Tageslicht von Grün unterscheidbar bleibt, ist deshalb ein eigener Prüfpunkt. Besonders wichtig ist das für die Muster von `LANDING` (grün, 1 Hz) und `STOP` (gelb,

1 Hz), die sich nur in der Farbe unterscheiden. Die übrigen Muster unterscheiden sich zusätzlich durch ihre Bewegung, also statisch, blinkend, im Wechsel oder als Lauflicht. Das hilft auch Beobachtern mit einer Rot-Grün-Sehschwäche, für die Grün, Gelb und Rot schwer zu unterscheiden sind.

Für die Bewertung sind folgende Randbedingungen festzuhalten. Jeder Kanal besteht aus zwei LEDs pro Bein, an der gesamten Drohne also aus acht LEDs pro Kanal. Ein Pegelwandler für die Datenleitungen ist nicht verbaut, die LEDs werden direkt mit dem 3,3 V-Pegel des Raspberry Pi angesteuert.

5.3.4 Übertragbarkeit auf den Flugbetrieb

Ein Test der Statusanzeige im Flugbetrieb wurde nicht durchgeführt und ist mit dem aktuellen Aufbau nicht möglich. Wie in Abschnitt 4.2.5 beschrieben, bietet derzeit kein Betriebssystem beides: Auf der SD-Karte für den Flug mit Ubuntu fehlt das Gerät `/dev/pio0`, auf der SD-Karte mit Raspberry Pi OS fehlt der Flug-Stack. Hinzu kommt, dass der LED-Knoten in keiner Startkonfiguration des Teams (`autonomy_master`) eingetragen ist und im Flug deshalb nicht automatisch starten würde.

Für einen Flugtest wären deshalb zwei Schritte nötig. Zunächst müssen die LEDs auch unter dem Flug-Stack ansteuerbar sein. Dafür kommen drei Wege in Frage:

SPI statt PIO
: Die Datenleitungen werden auf die MOSI-Pins der SPI-Schnittstellen umgelötet, und es wird eine neue Treiberklasse geschrieben.

Mikrocontroller
: Ein Mikrocontroller, zum Beispiel ein RP2040, wird per USB zwischen Raspberry Pi und LEDs geschaltet und erzeugt das Signal für die Streifen.

Flug-Stack auf Raspberry Pi OS
: Der Flug-Stack wird ebenfalls per Docker auf Raspberry Pi OS betrieben, das den PIO-Treiber bereits enthält.

In allen drei Fällen ändert sich nur die Treiberklasse hinter der Schnittstelle `LedStripDriver`. Zuordnung, Decoder und LED-Knoten bleiben unverändert, sodass die Ergebnisse der automatisierten Tests und der Großteil des Funktionstests weiterhin gültig sind. Hier zeigt sich der Nutzen der Schichtenarchitektur aus Abschnitt 4.2.3. Im zweiten Schritt muss der LED-Knoten in die Startkonfiguration des Teams aufgenommen werden.

Ein Flugtest könnte anschließend genau die Lücken schließen, die die SIL-Testumgebung offen lässt. Er würde zeigen, ob die echte Zustandsmaschine die getesteten Kombinationen im Flug tatsächlich erzeugt, wie sich die Anzeige unter dem Zeitverhalten des echten Systems verhält und ob die Zustandsmaschine bei einem Absturz von MAVROS den Fehler `FC_ERROR` rechtzeitig setzt.

6 Ergebnisse

In diesem Kapitel werden die Ergebnisse der in Kapitel 5 beschriebenen Prüfungen dargestellt. Die Gliederung folgt dabei der Evaluation, sodass jedem Abschnitt dort ein Abschnitt mit den Ergebnissen gegenübersteht. Abschließend werden die Ergebnisse den Anforderungen gegenübergestellt und die Grenzen der entwickelten Lösungen diskutiert.

6.1 Ergebnisse des Frames

6.1.1 Masse

6.1.2 Maßhaltigkeit und Montage

6.1.3 Mechanische Belastbarkeit

6.1.4 Fertigungsaufwand

6.1.5 Wartbarkeit

6.1.6 Flugtests

6.2 Ergebnisse der Statusanzeige

6.2.1 Automatisierte Tests der Anzeigelogik

Die Testsuite aus Abschnitt 5.3.1 wurde während der Entwicklung fortlaufend ausgeführt, insbesondere nach jeder größeren Änderung an der Anzeigelogik. Nach der letzten Überarbeitung der Anzeigemuster wurden von den ausgeführten Testfällen 292 bestanden, einer wurde übersprungen, und keiner ist fehlgeschlagen. Vor dieser Überarbeitung waren es 262 bestandene Testfälle. Die Differenz entspricht den Tests, die für die neuen Muster hinzugekommen sind. Beide Regressionstests für REQ-FUN-070 und für den Absturz von MAVROS wurden bestanden.

Übersprungen wurde ein Kompatibilitätstest mit dem Nachrichtenparser der Bodenstationsoberfläche des Teams. Dieser Test setzt voraus, dass das Paket der Oberfläche installiert ist, was auf dem Testrechner nicht der Fall war. Dass der LED-Knoten das Nachrichtenformat der Zustandsmaschine korrekt liest, ist davon unabhängig durch den Test mit dem echten Encoder belegt.

6.2.2 Funktionstest an der Hardware

Die Hardware wurde während der Arbeit schrittweise in Betrieb genommen und getestet. Zunächst wurden Pinbelegung und Farbreihenfolge mit `led_selftest` an der Drohne bestätigt. Nach der Überarbeitung der Anzeigemuster wurde der neue Stand auf den Raspberry Pi gebracht und mit der Testmatrix aus Tabelle 5.2 geprüft. Tabelle 6.1 zeigt das Ergebnis je Zeile. Alle bisher geprüften Muster entsprachen dem erwarteten Verhalten, und das Lauflicht lief in der richtigen Richtung über die Beine.

Tabelle 6.1: Ergebnisse des Funktionstests in der SIL-Testumgebung, Zeilennummern nach Tabelle 5.2

Nr.	Geprüfter Zustand	Ergebnis
1	Ausgangszustand nach `sil reset`	wie erwartet
2	`READY`, grünes Lauflicht	wie erwartet
3	`ARMED`	wie erwartet
4	`ARMED` im Modus `OFFBOARD`	offen
5	`TAKEOFF`	wie erwartet
6	`HOVER` im Modus `AUTO.LOITER`	offen
7	`APPROACHING` im Modus `OFFBOARD`	offen
8	`LANDING`	wie erwartet
9	`STOP` mit Missionsfehler `ABORT`	wie erwartet
10	Notlandung im Modus `AUTO.LAND`	offen
11	Systemfehler im Modus `MANUAL`	offen
12	Systemfehler `FC_ERROR`	wie erwartet
13	keine Verbindung zum Flugcontroller	offen
14	Zustandsgeber verstummt	offen
15	Beenden der Testumgebung	offen

Laut Protokoll des LED-Knotens zeigte die Anzeige direkt nach dem Start einmal das Signal für fehlende Daten, da zu diesem Zeitpunkt noch keine Nachricht angekommen war. Danach stand sie ruhig im Zustand `REST`, und nur die Navigationslichter leuchteten. Auch das entspricht dem Verhalten aus Tabelle 4.5.

Noch offen sind vor allem die Zeilen mit aktivem Autonomiekanal (4, 6, 7, 10) sowie Zeile 11. Gerade die Zeilen 10 und 11 belegen REQ-FUN-070 an der Hardware. In den automatisierten Tests ist dieses Verhalten bereits nachgewiesen, der Nachweis an den echten LEDs steht aber noch aus.

6.2.3 Sichtbarkeit und Unterscheidbarkeit

Der Sichtbarkeitstest wurde bisher nicht durchgeführt. Ob die Anzeige REQ-DES-060 und REQ-ENV-040 erfüllt und ob Gelb bei Tageslicht von Grün unterscheidbar bleibt, ist deshalb noch nicht belegt.

6.2.4 Übertragbarkeit auf den Flugbetrieb

Ein Test im Flugbetrieb wurde aus den in Abschnitt 5.3.4 genannten Gründen nicht durchgeführt. Alle Ergebnisse zur Statusanzeige stammen damit aus den automatisierten Tests und der SIL-Testumgebung. Sie belegen das Verhalten der Anzeige für die vorgegebenen Zustände, aber nicht, ob die echte Zustandsmaschine diese Zustände im Flug so erzeugt.

6.3 Diskussion

6.3.1 Erfüllung der Anforderungen

Tabelle 6.2 fasst zusammen, inwieweit die Anforderungen nach dem aktuellen Stand der Tests erfüllt sind.

6.3.2 Probleme und Grenzen

Bei der Statusanzeige ist die wichtigste Grenze, dass sie bisher nicht im Flugbetrieb eingesetzt werden kann (Abschnitt 6.2.4). Darüber hinaus wurden folgende Probleme festgestellt:

Tabelle 6.2: Erfüllung der Anforderungen nach aktuellem Stand

Anforderung	Ergebnis	Status
REQ-FUN-070	In den automatisierten Tests nachgewiesen, an der Hardware stehen die Zeilen 10 und 11 noch aus	teilweise
REQ-DES-150	Getrennte Datenleitungen je Bein, Fehlerisolation in den Tests nachgewiesen	erfüllt
REQ-OPS-090, REQ-OPS-100	Zeile 4 der Testmatrix noch nicht geprüft	offen
REQ-DES-060	Sichtbarkeitstest nicht durchgeführt	offen
REQ-ENV-040	Sichtbarkeitstest nicht durchgeführt	offen

Unbekannter Flugmodus wird als `AUTO.LOITER` gemeldet
Die Zustandsmaschine setzt einen PX4-Modus, den sie nicht kennt, auf `AUTO.LOITER`. An der Anzeige erscheint ein unbekannter Modus deshalb als autonom und nicht wie vorgesehen als unbekannt. Der rote Zweig des Autonomiekanals greift nur bei einem unbekannten Wert, den die Zustandsmaschine nie sendet. Auch die Übergangsbedingungen der Zustandsmaschine verlassen sich auf diesen falschen Wert und können nach `HOVER` wechseln. Eine Lösung wäre ein eigener Wert `MODE_UNKNOWN` im gemeinsamen Paket mit den Aufzählungstypen. Am LED-Code wäre dafür keine Änderung nötig. Da das gemeinsame Paket nur in Abstimmung mit dem Verantwortlichen der Bodenstation geändert werden darf, ist diese Lösung noch offen.

Fehlender Pegelwandler
Die LEDs werden direkt mit dem 3,3 V-Pegel des Raspberry Pi angesteuert, obwohl für den Datenpegel mindestens 3,5 V vorgesehen sind. In den bisherigen Tests funktionierte die Ansteuerung, eine sichere Übertragung ist so aber nicht gewährleistet.

Unterscheidbarkeit der Farben
Die Muster von `LANDING` und `STOP` unterscheiden sich nur in der Farbe, Grün gegen Gelb. Für Menschen mit Rot-Grün-Sehschwäche hilft dort nur der Kontext, also ob die Drohne sinkt oder die Position hält.

Kein automatischer Start
Der LED-Knoten ist in keiner Startkonfiguration des Teams eingetragen und würde im Flug nicht automatisch starten.

Keine Überwachung des LED-Knotens
Es gibt keinen Watchdog. Stürzt der LED-Knoten ab, bleibt das letzte Bild auf den LEDs stehen.

Lizenzen
Die verwendete PIO-Bibliothek steht unter der Lizenz GPL-2, das LED-Paket unter Apache-2.0. Vor einer Weitergabe des Pakets muss die Vereinbarkeit geklärt werden.

Die Aussagekraft der SIL-Testumgebung ist außerdem begrenzt, wie in Abschnitt 4.2.5 beschrieben. Sie belegt das Verhalten der Anzeige, aber nicht das Zusammenspiel mit der echten Zustandsmaschine im Flug.

7 Zusammenfassung und Ausblick

7.1 Zusammenfassung

7.2 Ausblick

Literatur

[1] .

[2] .

[3] Adobe Stock. *Motor Order Diagrams of the Quad Drone or Quadcopter. Set of Vector Infographics of Airframes and Types Quad X, Quad H, Quad V and Quad P*. Bild-ID 243287639. URL: https://stock.adobe.com/vn/images/motor-order-diagrams-of-the-quad-drone-or-quadcopter-set-of-vector-infographics-of-airframes-and-types-quad-x-quad-h-quad-v-and-quad-p/243287639 (besucht am 30. 10. 2026).

[4] Allied. *What is FDM 3D Printing?* URL: https://alliedcn.com/what-is-fdm-3d-printing/ (besucht am 30. 10. 2026).

[5] Beate Beermann u. a. "Veränderungen in der Arbeitswelt - Reflexion des Arbeitsschutzsystems". de. In: (2020), S. 10.

[6] *Bericht „Wohnen am Campus I"*. URL: https://www.adlershof.de/fileadmin/user_upload/ebooks/WaC/index.html#p=36 (besucht am 11. 01. 2022).

[7] Bonvaero. *Quadcopter Drones: The Ultimate Guide*. URL: https://bonvaero.com/quadcopter-drones/ (besucht am 30. 10. 2026).

[8] Heidrun Friese u. a. *Handbuch Soziale Praktiken und Digitale Alltagswelten*. de. Google-Books-ID: rRDUDwAAQBAJ. Springer-Verlag, März 2020. ISBN: 978-3-658-08357-1.

[9] Felix Gauger, Andreas Pfnür und Jan Skarabi. *Arbeitswelten im Wandel: Coworking Spaces. Eine empirische Befragung der Eigenschaften und Nutzerpräferenzen von Coworking Spaces*. ger. Arbeitspapiere zur immobilienwirtschaftlichen Forschung und Praxis 39. Darmstadt: Technische Universität Darmstadt, Forschungscenter Betriebliche Immobilienwirtschaft, 2020. URL: http://hdl.handle.net/10419/223375.

[10] Jens Libbe u. a. *Memorandum Post-Corona-Stadt; Für eine suffiziente und resiliente Entwicklung von Städten und Regionen*. Techn. Ber. Juli 2020.

[11] Ashish Mishra. *Different Types of Drone Frames: Monocopter to Octocopter*. Circuit Digest. 27. Dez. 2021. URL: https://circuitdigest.com/article/different-types-of-drone-frames-monocopter-to-octocopter (besucht am 30. 10. 2026).

[12] oneclick. *VDI und DaaS: Vor- und Nachteile*. 5. Juni 2019. URL: https://oneclick-cloud.com/de/blog/trends/vdi-und-daas-vor-und-nachteile/ (besucht am 23. 04. 2023).

[13] Peterson. *Quadcopter vs Hexacopter*. Rc Things. URL: https://rcthing.com/quadcopter-vs-hexacopter/ (besucht am 30. 10. 2026).

[14] Glenn Riney. *Hexacopter vs Quadcopter: What's the Difference and How to Choose?* Extreme Fliers. 16. Sep. 2026. URL: https://www.extremefliers.com/hexacopter-vs-quadcopter/ (besucht am 30. 10. 2026).

[15] *Studie zu Homeoffice und Corona - Beschäftigte fürchten um Karrierechancen*. de. URL: https://www.deutschlandfunk.de/studie-zu-homeoffice-und-corona-beschaeftigte-fuerchten-um-100.html (besucht am 01. 12. 2021).

[16] Unmanned Tech. *Quadcopter Frame Design: H-Frame vs X-Frame vs True-X Configuration*. URL: https://www.unmannedtechshop.co.uk/de/blogs/knowledge-base/quadcopter-frame-design-h-frame-vs-x-frame-vs-true-x-configuration (besucht am 30. 10. 2026).

[17] WERMA Signaltechnik GmbH + Co. KG. *Signalisierungslösungen für Maschinen und Anlagen*. URL: https://www.werma.com/produkte/signalsaeulen/ (besucht am 30. 10. 2026).

A Anhang